

Is content-based filtering more effective than collaborative filtering in recommender systems when dealing with a small user base?

Authors

Owen Gombas

University of Bern

Arthur Gygax

University of Neuchâtel

Table of contents

1	Introduction	4
1.1	Problem statement	4
1.2	What do we mean by "small"	4
1.3	Report vocabulary	5
2	Data collection	6
2.1	From the Spotify API	6
2.1.1	User data retrieval	6
2.1.2	Retrieval of tracks that are not related to users	7
2.1.3	Spotify forbidding AI training	7
2.2	Data Generation	8
2.2.1	Generated datasets	10
2.3	Available Datasets	11
3	Data Analysis	12
3.1	User based analysis	12
3.1.1	Tracks quantity per user	12
3.1.2	User genre quantity	13
3.1.3	User features distribution	14
3.1.4	User interactions	15
3.1.5	User features pairplot	16
3.1.6	User PCA 2D	16
3.2	Genre based analysis	17
3.2.1	Quantity of (very) specific genres	17
3.2.2	Quantity of genres	17
3.2.3	Genre features distribution	19
3.2.4	Genre scatter matrix	20
3.3	The whole dataset	21
4	Affinity inference on Spotify data	22
4.1	Top tracks	22
4.2	Liked tracks	22
4.3	Playlists tracks	23
4.4	Final values	23
4.5	Unknown tracks	23
4.6	Loading the data	24
5	Metrics	25
5.1	Vector Set Divergence (VSD)	25
5.1.1	Setting up the ground truth	26
5.1.2	Dealing with multiple users	26
5.2	MPR, NDCG and Recall	26
6	Content-based filtering using Nearest Neighbors	27

6.1	Expected results and thoughts	27
6.2	Key difference with collaborative-filtering methods	28
6.3	Data processing	28
6.3.1	Loading the data	28
6.3.2	Dropping the duplicates {"username", "id"}	28
6.3.3	Scaling the numerical features	28
6.3.4	Splitting the user sets	28
6.4	Model selection	29
6.4.1	Grid Search	29
6.4.2	Test	30
6.5	Inference	30
7	Collaborative filtering with implicit feedback	31
7.1	Notations	31
7.2	Data processing	31
7.2.1	Loading the data, dropping the duplicates and scaler	31
7.2.2	Keeping the tracks that appears at least N times	32
7.2.3	Splitting the user set	32
7.2.4	Building the interaction matrix	33
7.2.5	Scaling the matrix entries	34
8	Logistic matrix factorization	35
8.1	General probabilistic framework	35
8.2	Our framework	35
8.3	Learning the parameters	36
8.4	Prediction	37
8.5	Training process and model selection	38
8.6	Latent space visualization	40
9	Variational Autoencoders	42
9.1	The general model	42
9.2	Variational inference	43
9.3	Learning the parameters	44
9.4	Prediction	46
9.5	Training process and model selection	46
9.6	Latent distribution visualization	49
10	Ethical discussion	50
10.1	Privacy, anonymization method and ethical discussion on our project	50
10.2	Ethical discussion on recommender systems	50
11	Results and discussions	52
11.1	Results	52
11.2	Model selection	52
11.3	NN vs NN Full	53
11.4	Best Nearest Neighbor hyper-parameters	53
11.5	Discussion	53

12 Future perspectives and improvements	55
12.1 Sparse matrix support for large batches	55
12.2 Human based metrics	55
12.3 Individual improvements of the models	56
12.4 Metric compatibility	56
13 Developer documentation	57
14 Conclusion	58
15 Bibliography	59

1. Introduction

Over the past few years, recommender systems have become part of our everyday life, for the vast majority they are part of web services we interact with everyday. Whether in e-commerce, where they suggest articles aligning with buyers' interests; on social media platforms, where they are used to suggest friends, pages, or content based on users' past interactions and interests. Or in music streaming services where they are used to build personalized playlists, recommend new songs, and suggest artists based on users' listening history and preferences. These systems generally rely on either or both two different paradigms: collaborative filtering and content-based filtering.

Collaborative Filtering: Collaborative-Based Filtering predicts user preferences by exploiting information about many other users. It analyzes interactions and behaviors of a group of users to make predictions about the preferences of a specific user, with the main assumption that if two users share the same affinity on an item, they are more likely to also share the same affinity on another item. In general, collaborative filtering requires a large amount of users.

Content-Based Filtering: In contrast, Content-Based Filtering approach uses the features of the items belonging to a user to produce personalized recommendations. By analyzing the proximity of a large amount of items that a user didn't interact with, with the users preferences. It simply suggests items similar to those the user has already shown interest in, focusing only on the content's features.

In the following sections, we're going to take a closer look at how we build collaborative and content-based recommendation systems especially for Spotify. We'll explore the specific methods, algorithms, and evaluation techniques that make it all work.

1.1 Problem statement

With the number of new songs available on Spotify always increasing, it has become a challenge for users to discover new songs that satisfy their tastes. The objective of this report is to implement different recommendation systems and understand in details how they work. We choose Spotify to be our data source for this project but these techniques can be applied in a more general way and can be used with different type of items, not only music songs.

1.2 What do we mean by "small"

We could only retrieve a user base of 8 users, which is a very (very) small amount of users when dealing with collaborative filtering models, however we could retrieve around 10'000 tracks. So we asked ourselves if that's enough to build an efficient collaborative-based recommender systems which would outperform a content-based filtering recommender system with a large amount of tracks data. [Johnson \(2014\)](#) is dealing with a user base of 50'000 users with the tracks of the 10'000 most popular artists of Spotify. In the other hand, [Liang et al. \(2018\)](#) uses publicly available datasets [MovieLens 20M Dataset](#) which contains 138'000 users, [Netflix Prize Data](#) which contains 480'189 users and [Million Song Dataset](#) which has 1'019'318 users. Taking all that into account, and considering the fact that we do not possess a great amount of

computational power, we will set the threshold to consider a relatively small number of users below 150.

1.3 Report vocabulary

Please note that in this work we refer to music songs with different terms: songs, tracks, items but it all means the same. The same goes for dataset and DataFrame.

2. Data collection

2.1 From the Spotify API

2.1.1 User data retrieval

Spotify has a public REST API that allows developer to retrieve useful information about the tracks and the Spotify users, such as their playlists, their liked tracks and also their top listened tracks for three different time ranges (with a maximum of 50 tracks per time range):

- **long_term**: Calculated from several years of data and including all new data as it becomes available
- **medium_term**: Approximately last 6 months
- **short_term**: Approximately last 4 weeks

For our data retrieval process we used these following API endpoints with [Spotipy](#)¹ to build our dataset:

- <https://developer.spotify.com/documentation/web-api/reference/get-users-top-artists-and-tracks>
- <https://developer.spotify.com/documentation/web-api/reference/get-list-users-playlists>
- <https://developer.spotify.com/documentation/web-api/reference/get-users-saved-tracks>
- <https://developer.spotify.com/documentation/web-api/reference/get-several-audio-features>

To retrieve the data it works as the following procedure:

1. Ask the person for the email they used to create their Spotify account.
2. We add this e-mail to the authorized user list in our Spotify Developer app dashboard <https://developer.spotify.com/dashboard/>.
3. We generate an authentication URL that we send to the person.
4. By clicking on this URL the person is redirected to a Spotify login interface.
5. Once logged it, the person is redirected to the redirect URL with an authentication code. This new URL should be send back to us to complete the authentication process.
6. Once we got this URL we can effectively retrieve the person's Spotify data.

We first retrieve all the tracks known² by all eight users who accepted to give us their data. Once collected, the tracks do not directly have numerical features related to them, that's why we have to use the **get-several-audio-features** endpoint for every track to populate our database

¹ A Python module which simplify the API calls to the Spotify API

² Meaning that the track is included in either their top listened songs, playlists or liked songs

with useful features. From our 8 users, we could then retrieve 12'429 songs without deducting the duplicates (7'211 from playlists, 4'018 from liked songs and 1'200 from top listened songs). Which all have 14 features: *duration_ms*, *acousticness*, *danceability*, *energy*, *instrumentalness*, *liveness*, *loudness*, *mode*, *speechiness*, *tempo*, *time_signature*, *valence*, *release_year*, *popularity*.

2.1.2 Retrieval of tracks that are not related to users

To generate more creative and diverse recommendations, we should also include songs that are unknown to our users. That's why we manually selected playlists with the most diverse tracks, considering features such as; *release_year*, *popularity*, *genre* and *language*, and ensuring a large amount of tracks. We selected 8 large playlists from which we retrieve 7'893 tracks in total. The playlists we use can be accessed through the following links¹:

- 1960-2023:
<https://open.spotify.com/playlist/0iwMTcAhHhqv3ytEfhWs7>
- Every US number one (1960 - 2023):
<https://open.spotify.com/playlist/24P5rHoP6MhEiEaB25l1wI>
- top hits 2000-2024:
<https://open.spotify.com/playlist/7E3uEa1em0cbZJuB8sFXeK>
- Best Unknown Songs:
<https://open.spotify.com/playlist/0AVwU8Bp08p7MS5LeU8lmj>
- best unknown artists/songs:
<https://open.spotify.com/playlist/6RCtJw3jv1M03HFeeEY9hc>
- Best (mainly) unknown songs:
<https://open.spotify.com/playlist/2jqYP6k583TFLfejNZ2qhn>
- The Greatest Mixed Playlist Ever:
<https://open.spotify.com/playlist/5zwLw7HmUqTZAXwOTqIdqy>

2.1.3 Spotify forbidding AI training

Be aware that Spotify does not allow developers to use this data to train AI algorithms (cf. <https://developer.spotify.com/terms#section-iv-restrictions>). For this report, the only purpose of retrieving these data is to perform data analysis aiming to build an artificial data generator providing us with more control over our environment. Upon project submission the dataset is provided to the Professor, new users data can be added easily using the `retrieve_*.ipynb` notebooks, allowing for further analysis. We reject all responsibility for the use of this data, since we only provide what can be retrieved by anyone.

cf.

- `lib/spoti.py`: Manage the user connection and data retrieval
- `retrieve_user_auth.py`: Retrieve an URL allowing our app to retrieve user's data

¹ If they aren't available anymore online, they can be retrieved from our dataset

- `retrieve_liked_tracks.ipynb`: Retrieve the tracks the users liked
- `retrieve_playlists.ipynb`: Retrieve the users tracks contained in their playlists
- `retrieve_top_tracks.ipynb`: Retrieve the tracks the users listened to the most
- `retrieve_public_playlists.ipynb`: Retrieve the tracks contained in playlists (not related to any users)

2.2 Data Generation

To address our initial question, we then developed a data generator to build similar datasets from the one we could get from Spotify. This methodology also provides us with better control over the quantity of our data, which is crucial, in particular for the collaborative-filtering based recommender systems we use.

We generate N datasets D_u (each of these dataset can have a different size) which have a given¹ set of numerical features that follows a given probabilistic distribution², each of these N dataset are assigned to an artificial username that is represented by a name, the same goes for the items of this dataset, we assign every row to a random song name, we used Faker³ to generate these random names. These N datasets are merged into one bigger dataset that has size $|D_1| + \dots + |D_N|$, with N users and each user have $|D_u|$ tracks. For each of the entries, we also generate an affinity score that ranges from 0 to 1. The features it generates are the same 14 features we have in the Spotify dataset and are sampled from a normal distribution ($\mu = 1.0$ $\sigma = 0.1$).

¹ In the argument of the `ArtificialTrackDataset` constructor

² Given in the `ArtificialTrackDataset.randomize` method

³ A library to generator random values of different nature: <https://faker.readthedocs.io/en/master/>

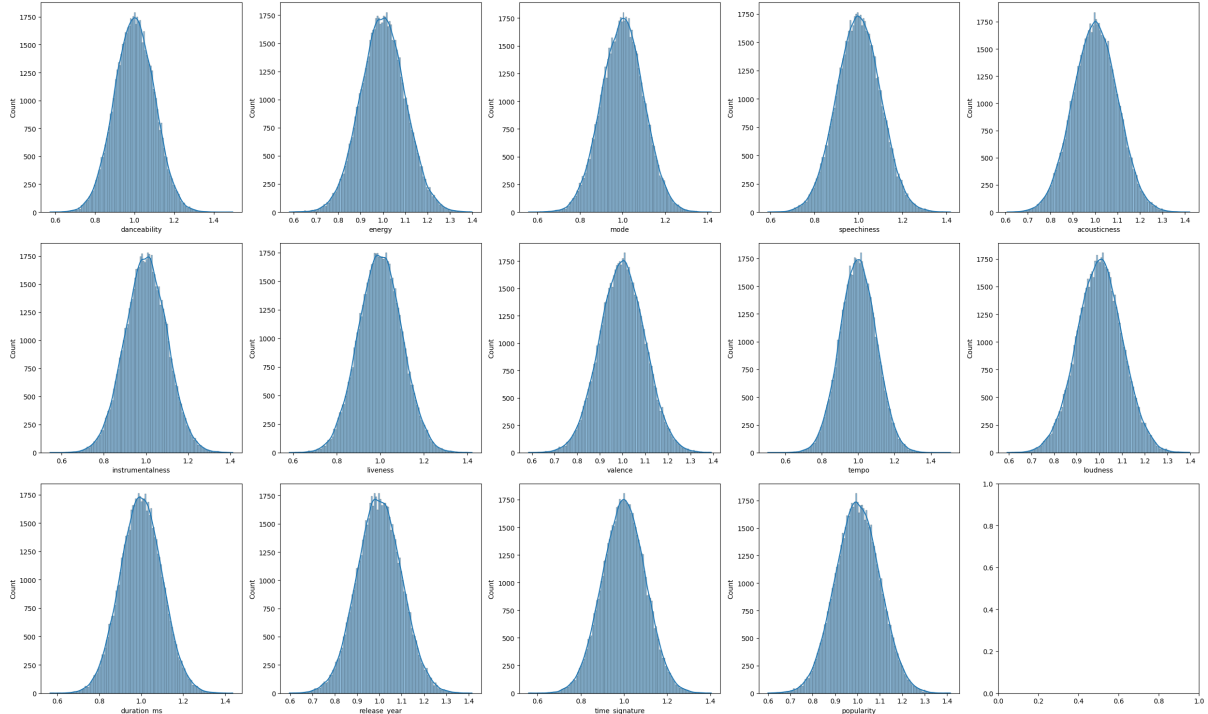


Figure 2.1: generated_data_distribution.png - An example of the features distribution for a generated dataset

Once this dataset is generated no tracks are shared by any user because it was independently generated. We have to mix the entries of the dataset, for that purpose we implemented the `generate_dataset_with_similarities` method that takes in argument `mix_with_someone_else_chance` which tells the algorithms the probability (from 0 to 1) that a user A can mix songs with another different user B . If the mix has to be performed `min_track_mixing_percentage` and `max_track_mixing_percentage` are the arguments that tell how much of the user A 's tracks will replace user B 's tracks so they share common songs. To give more chance to lower mixture percentage, we pick a random value from a [beta distribution](#) ($\alpha = 1, \beta = 4$) and scale it from `min_track_mixing_percentage` to `max_track_mixing_percentage`. The affinity is regenerated so the track T_i has a different affinity for user A than for user B .

All the generated features of our dataset follows a normal distribution (loc=0, scale=1) and modified to be bounded by 0 and 1.

cf. `lib/artificial_data.py`, `artificial_data.ipynb`

2.2.1 Generated datasets

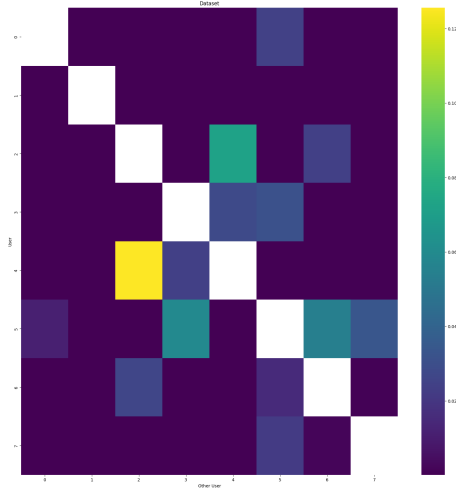


Figure 2.2: user'mat'8.png - Tracks sharing heatmap of 8 users

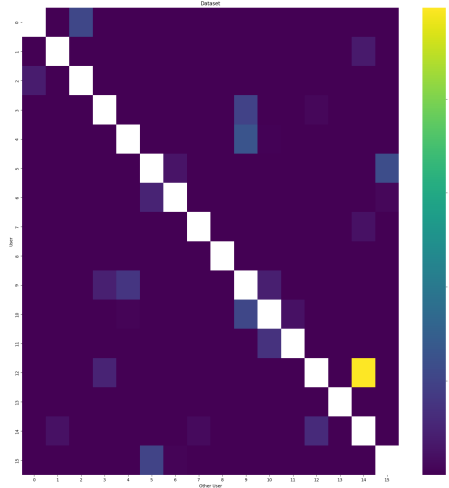


Figure 2.4: user'mat'16.png - Tracks sharing heatmap of 16 users

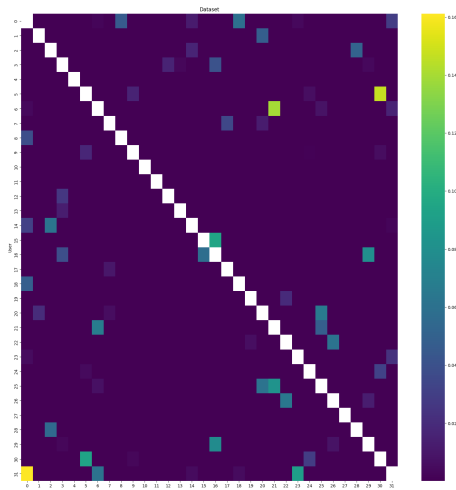


Figure 2.3: user'mat'32.png - Tracks sharing heatmap of 32 users

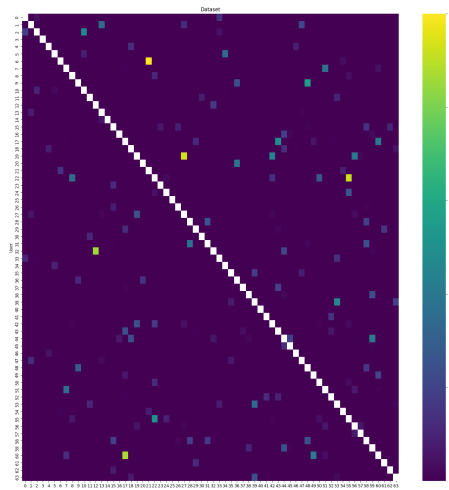


Figure 2.5: user'mat'64.png - Tracks sharing heatmap of 64 users

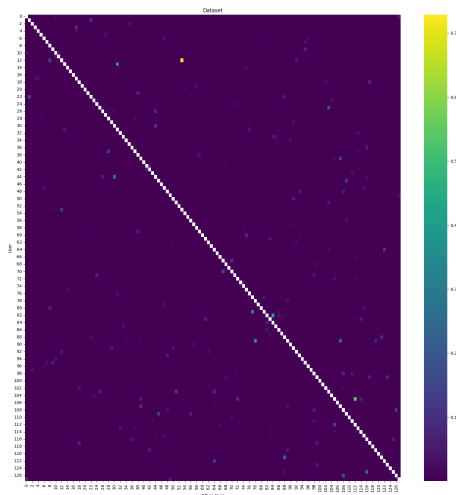


Figure 2.6: user`mat`128.png - Tracks sharing heatmap of 128 users

2.3 Available Datasets

Some datasets are openly available online and well suited for collaborative-filtering based recommender systems such as the MovieLens 20M Dataset¹. These dataset are way bigger than the ones we generated or retrieved from Spotify. After processing they can build a dense Matrix for collaborative filtering that could use more than 9GB of memory. Since these interactions matrices a mainly formed of zeros, some implementation uses SciPy sparse² to store these matrices without using too much memory. We tried to achieve the same using only PyTorch but at the time we write this report their implementation of sparsity is still in beta so we have some errors while implementing the sparsity in our algorithms (we tried using CPU and MPS devices). To be able to finish our project on time we didn't spend more time trying to implement the sparsity hoping that in a near future we can slightly adapt our PyTorch implementation to use it.

¹ <https://grouplens.org/datasets/movielens/20m/>

² <https://docs.scipy.org/doc/scipy/reference/sparse.html>

3. Data Analysis

We performed two preliminary analysis on the data we retrieved from Spotify, one concerning the users and the other one for the genres of the tracks.

All the people who allowed us to use their information have their name anatomized in our datasets and in this report. We generated fake names to represent them, however they all allowed us to keep a trace of the anonymization process so we can verify our further potential analysis. The document that stores the correspondences of the generated name and the real name is confidential and will not be reviled without the consent of all the participants.

Some of the following plots are interactive, for further analysis we recommend you to refer to the following file: `data_analysis.user.ipynb`

3.1 User based analysis

3.1.1 Tracks quantity per user

Number of Liked Tracks per User

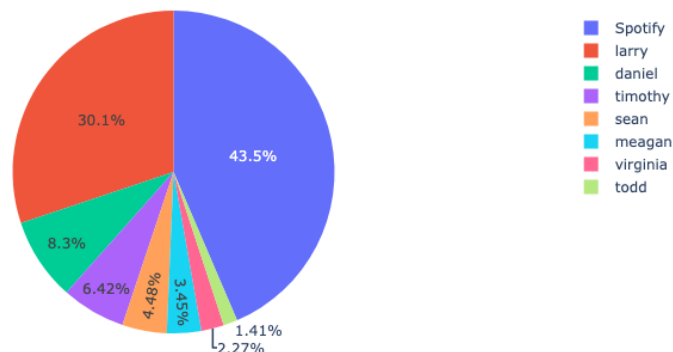


Figure 3.1: users'count.png - The quantity of tracks we have per user, "Spotify" represent all the tracks that we retrieve from public playlists [2.1.2](#)

3.1.2 User genre quantity

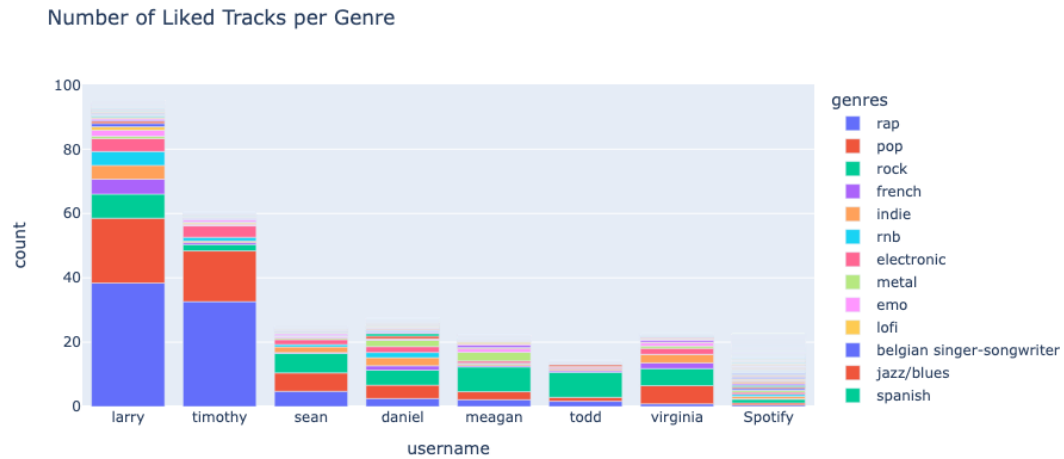


Figure 3.2: users'genres.png - How much every user listen to different genres, "Spotify" represent all the tracks that we retrieve from public playlists[2.1.2](#)

3.1.3 User features distribution

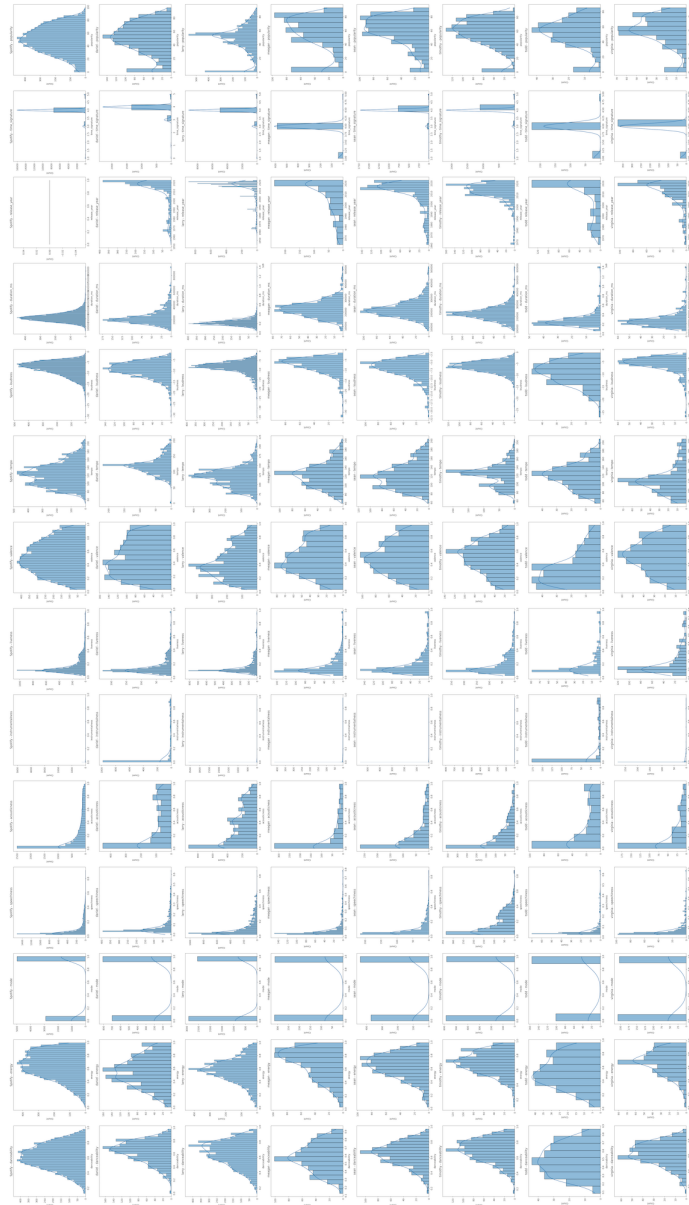


Figure 3.3: users'distribution.png - It shows the discrete distributions for every user on all the features we retrieved from the Spotify API, "Spotify" represent all the tracks that we retrieve from public playlists [2.1.2](#)

3.1.4 User interactions

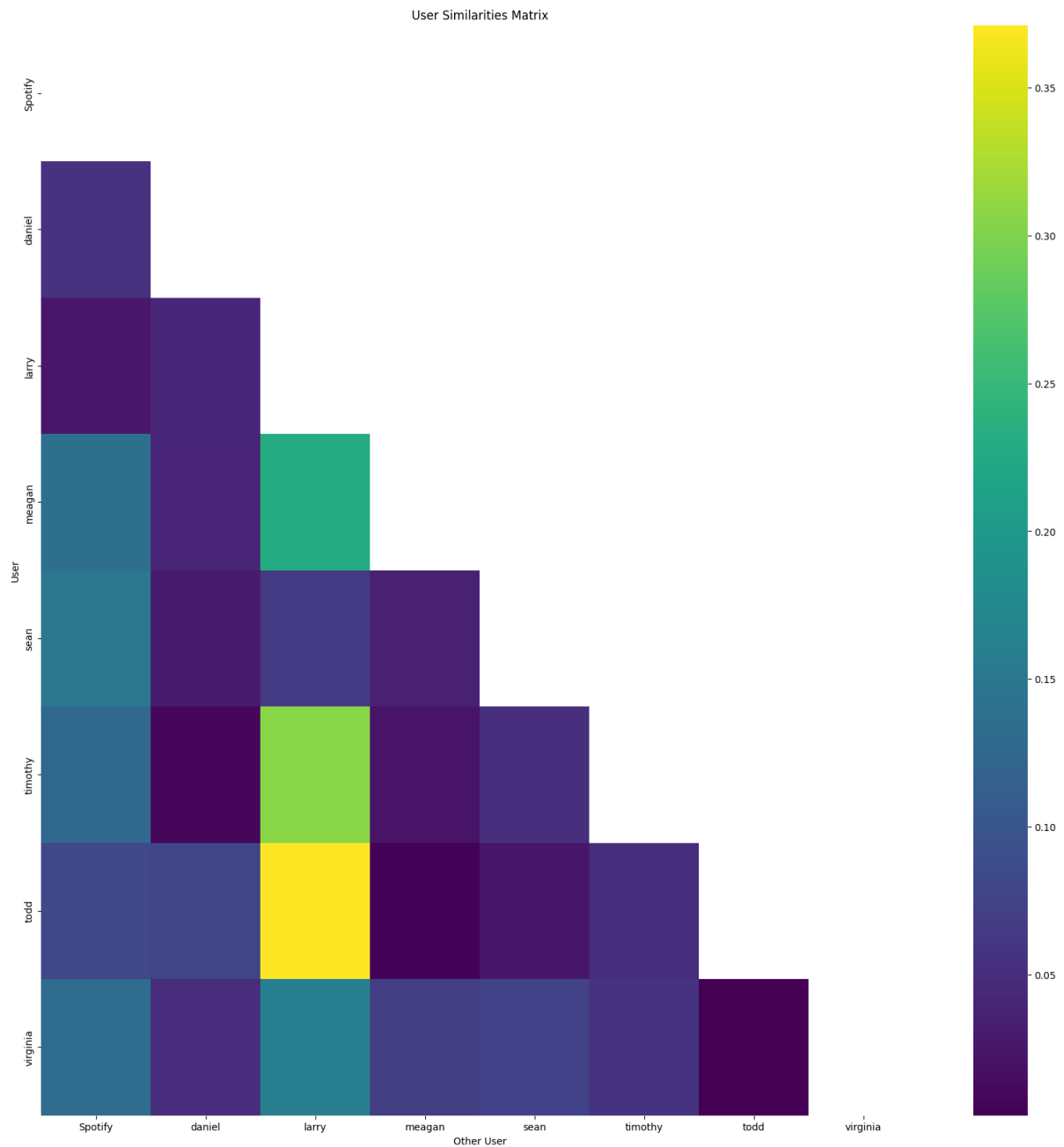


Figure 3.4: users'matrix.png - The interaction heatmap of a user A with a user B , it shows how much a user A shares tracks with an user B

3.1.5 User features pairplot

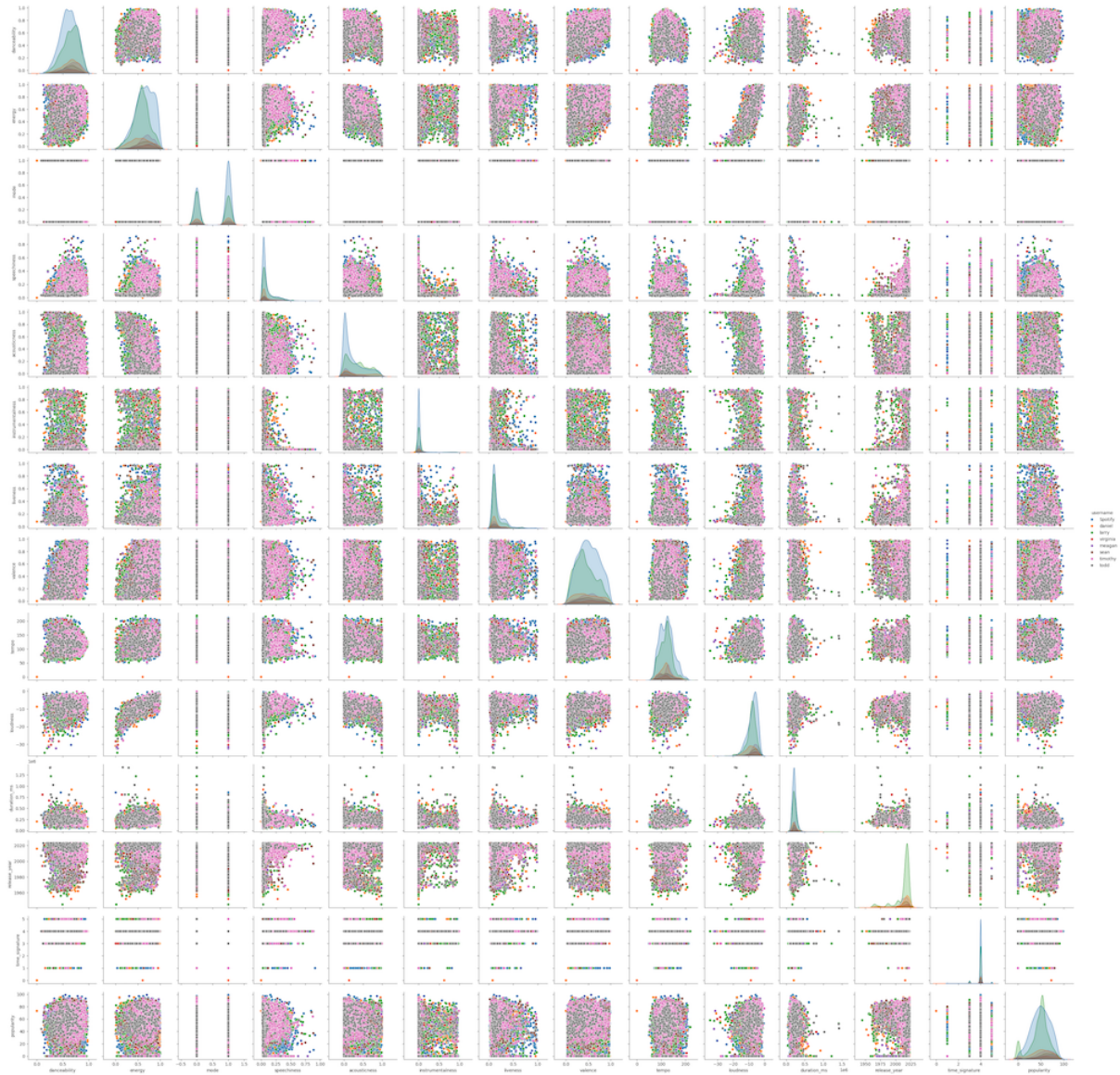


Figure 3.5: users'pairplot.png - For every user (here they are represented with individual colors) we can plot every feature against each other and to form a pairplot, this can be view as the raw representation of a user

3.1.6 User PCA 2D

To counter the fact that the pairplot isn't really readable we can perform a dimensional reduction on the data using PCA and plot it.

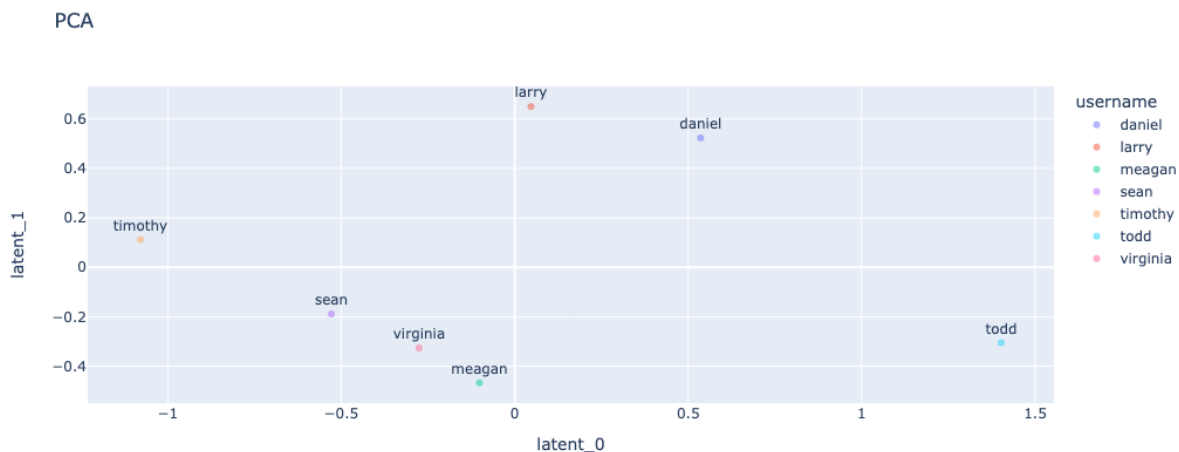


Figure 3.6: users'pca'2d.png - PCA to our features to 2 components so we can plot it (and show it in this report). If the dimensionality reduction isn't too low, it would be a great indicator on how much users are close to each other. You can use the following notebook to visual that PCA analysis for higher dimensions: `data_analysis_user.ipynb`

3.2 Genre based analysis

3.2.1 Quantity of (very) specific genres

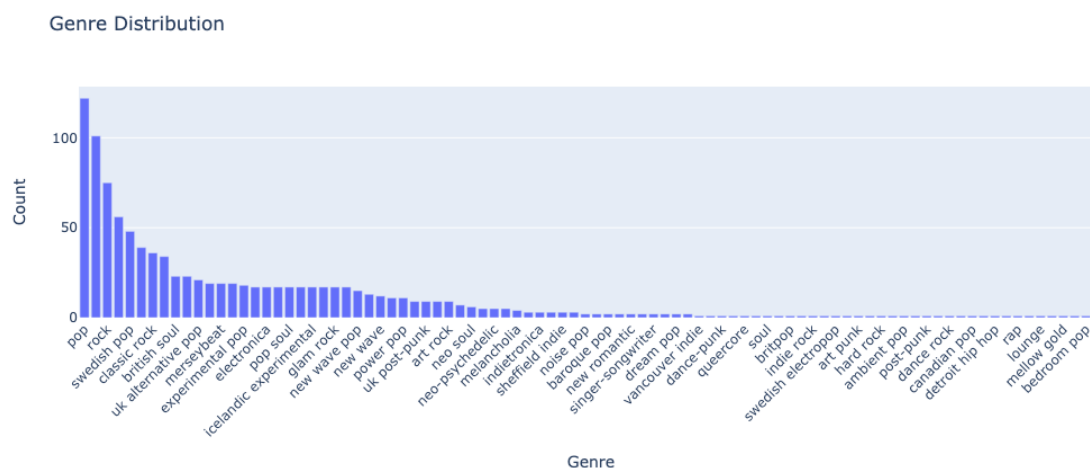


Figure 3.7: genres'quantity.png - The quantity of very specific genres we have in our dataset (cropped to take the most present ones)

3.2.2 Quantity of genres

We simplified the genres so the sub-genres of rock are wrapped into the rock genre, the same goes for rap which is almost the same as hip-hop. We performed this grouping for all the main

genres we have in our dataset to end up with more simple genres.

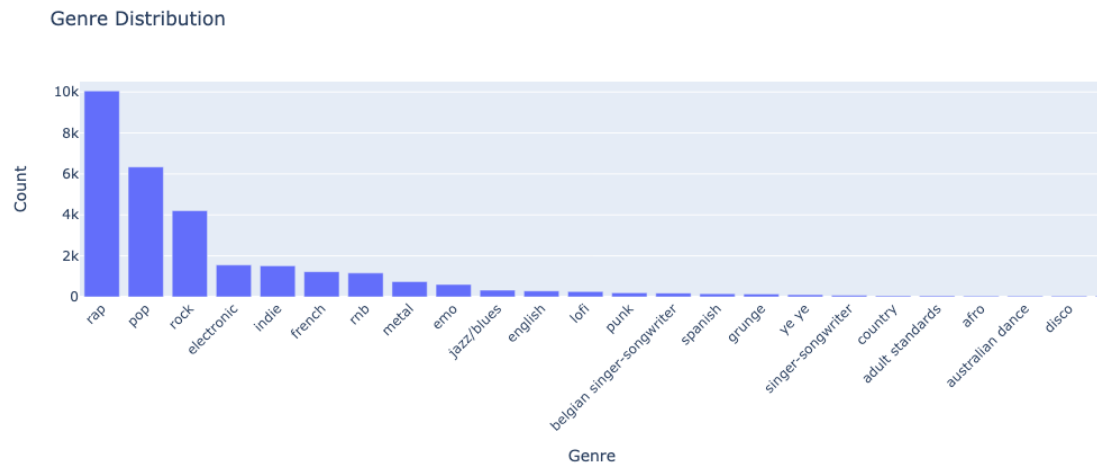


Figure 3.8: genres'normalized'quantity.png - The quantity of genres we have in our dataset (cropped to take the most present ones)

3.2.3 Genre features distribution

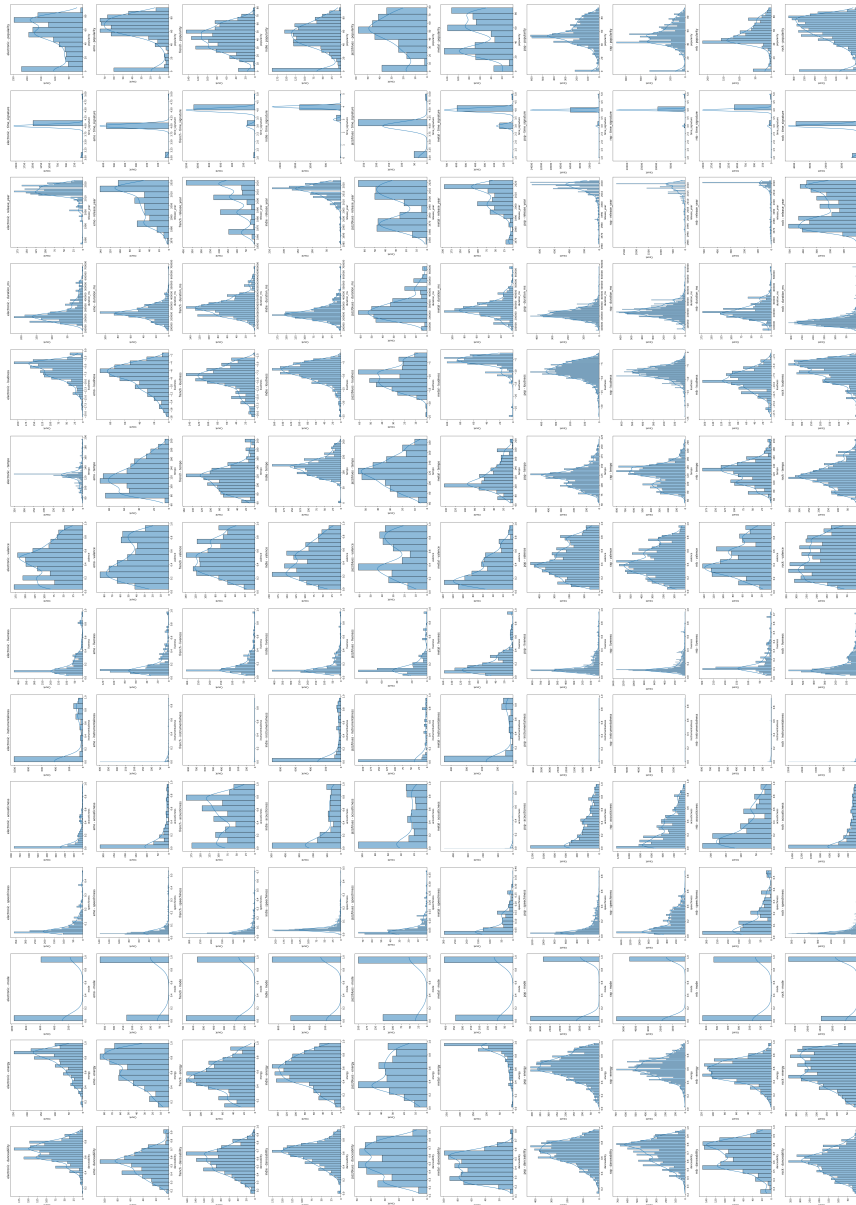


Figure 3.9: genres' features' distribution.png - It shows the discrete distributions for every genres on all the features we retrieved from the Spotify API



Figure 3.10: genres'scatter'matrix.png - For every feature we plot tracks against another feature, the color represents the genre. We can see some clusters forming.

3.3 The whole dataset

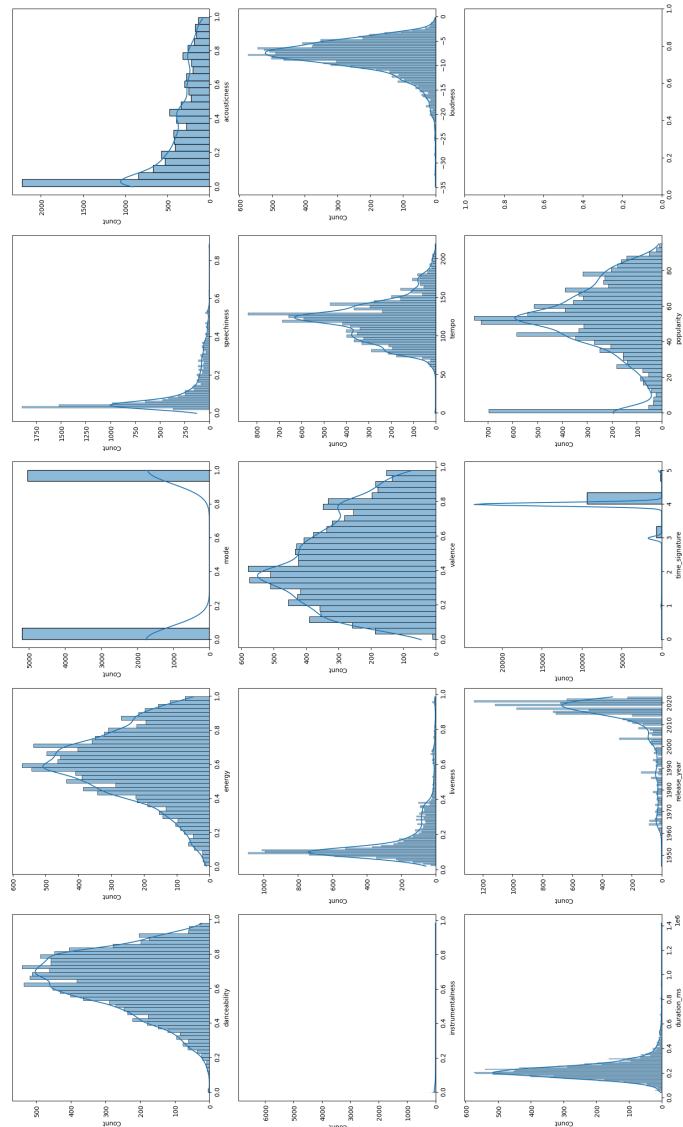


Figure 3.11: dataset'features'distribution.png - The features distribution from the whole dataset we have, including the tracks that are not related to any users [2.1.2](#)

4. Affinity inference on Spotify data

With our dataset from Spotify we do not get any explicit affinity score for a user and a track. However, we wondered if the order from which we receive the top listened tracks from the users is important and match the affinity, meaning that the first song of this list is the one the user like¹ the most. We first refereed to an issue opened on the Spotify Web API GitHub repository: <https://github.com/spotify/web-api/issues/1425> which didn't give a concrete answer. Then we checked an online service called "Stats for Spotify" which allows people to retrieve an ordered list of their top tracks/artists from the Spotify API, we compared this information to what we received from our own API calls using Spotify and it matched. To be sure we also performed a manual analysis with our own accounts and verified that it sorts the tracks according to what we listened to the most for these time ranges and it indeed matched the order from which we received them.

4.1 Top tracks

Based on this observation, we can define a decreasing function that attributes an affinity score according to the order of the tracks we get from this endpoint. We choose a simple linear decreasing function using `np.linspace(start, end, 50, endpoint=False)` but we have set different `start` value for each time range:

- **short_term:**
The affinity goes from `start=1.0` to `end=0.5` (not included), meaning that the first song has a 1.0 affinity score and it linearly decrease to 0.5 (not included) for the last song.
- **medium_term:**
The same logic goes for the `medium_term` but with `start=0.7` and `end=0.4`.
- **long_term:**
The same logic goes for the `long_term` but with `start=0.5` and `end=0.3`

Note that these values are hard-coded, for further and deeper model selection we would find the best values for these hyper-parameters. For this submission we couldn't find the time to implement that but it would be a great improvement.

4.2 Liked tracks

The liked tracks also have an interesting feature from which we can infer an affinity which is *added_at*. Assuming that a user is more likely to enjoy similar tracks to which they listened recently we can use the same logic as discussed for the top tracks in Section 4.1. We set a `start` and `end` value and assume all the liked items have an affinity based on the time they were added to the liked tracks. Then the most recent song added has an affinity which is equal to the `start` value.

¹ Determined by Spotify

4.3 Playlists tracks

We could also assign a constant affinity to the tracks contained in the playlists because no feature allows us to infer any affinity score. For instance, we can say that every songs that are in playlists have an affinity of 0.2. Again, this value can be fine tuned with a deeper model selection.

4.4 Final values

For this report the following chosen values are:

time_range	start	end
short_term	1.0	0.5
medium_term	0.7	0.4
long_term	0.5	0.3

Table 4.1: Affinity values for top tracks

subset	start	end
liked_tracks	0.7	0.1

Table 4.2: Affinity values for liked tracks

subset	value
playlist_tracks	0.2

Table 4.3: Affinity values for playlists tracks

A user can have the same song appearing in their top tracks, liked tracks and multiple playlists. As that would be reflected in our dataset, we have to drop duplicated on the **username** and **track id** keys. When dropping these duplicates, pandas will keep only the first occurrence and because of the order the tracks have in our dataset the first occurrence have highest affinity value so we do not loose any potential affinity information by reducing it to 0.

4.5 Unknown tracks

The tracks the a user doesn't know has a implicit 0 affinity with the user, this value is also used in the different paper we based our work on [Johnson \(2014\)](#) and [Liang et al. \(2018\)](#).

4.6 Loading the data

To load the data into a Python script or a Jupyter notebook we implemented a method called `load_all_tracks` that allows us to do it easily. We can pass the affinity inference parameters (start, stop for the tracks types) and retrieve our collected data. Here is an example of it's usage:

```
df = spoti.load_all_tracks(
    base_path=data_path.DATA_PATH,
    users=users_list.USERS,

    # Affinity inference parameters
    top_tracks_affinity_start = {
        "short_term": 1.0,
        "medium_term": 0.7,
        "long_term": 0.5,
    },
    top_tracks_affinity_end = {
        "short_term": 0.5,
        "medium_term": 0.4,
        "long_term": 0.3,
    },
    liked_tracks_start_affinity=0.7,
    liked_tracks_end_affinity=0.1,
    playlists_affinity=0.2,

    # [a, b] or [a, b[ interval
    top_tracks_include_end=False,
    liked_tracks_include_endpoint=False,

    # Load the tracks that aren't related to any user
    # (Referred further as the enriched data)
    load_spotify_tracks=False,
)
```

5. Metrics

5.1 Vector Set Divergence (VSD)

Since the different metrics we saw in the different papers MPR ([Johnson \(2014\)](#)), NDCG@K and Recall@K ([Liang et al. \(2018\)](#)) aren't quite compatible with our content based Nearest Neighbors recommender and difficult to adapt for that purpose. To be able to answer our initial scientific question we had to come up with a metric that allows us to compare content-based recommender systems and collaborative-based recommender systems.

As we didn't find an existing name for this metric, we will refer to it as the "**Vector Set Divergence**", the goal is to have a score close to 0 if two sets of vector are identical and the more these sets of vector are different the more this value would diverge from 0. To achieve that we use the `pairwise_cosine_similarity` method from [TorchMetrics](#) which computes the cosine similarity between every pairs of two collections of vectors. Then we use the [Hungarian algorithm](#) with [Scipy's linear_sum_assignment](#) to find the optimal assignment that pairs each vector in set A with a unique vector in set B such that the sum of distances for these pairings is minimized. We then choose to perform our model selection using this metric for all models.

However, please note that this metric is only used to answer our scientific question. To perform model selection on the individual type of model we would use the metrics described in their paper which we also successfully implemented.

VSD does not take into account the order of the items but instead it works on the values of the features of the items, allowing us to compare more easily the results of content-based and collaborative-based recommender systems.

```
from scipy.optimize import linear_sum_assignment
from torchmetrics.functional import pairwise_cosine_similarity

def vector_set_divergence(A: torch.Tensor, B: torch.Tensor):
    """
    Calculate the similarity score two matrices set of vectors

    Args:
        A (torch.Tensor): The first matrix.
        B (torch.Tensor): The second matrix.

    Returns:
        torch.Tensor: The similarity score.
    """
    # Create a cost matrix using cosine similarity
    cost_matrix = 1 - pairwise_cosine_similarity(A, B)

    # Find the optimal assignment (maximization) using the Hungarian algorithm
    row_ind, col_ind = linear_sum_assignment(cost_matrix.cpu().numpy())
```

```
# Calculate score
score = cost_matrix[row_ind, col_ind].sum()
return -score
```

5.1.1 Setting up the ground truth

To set up the ground truth to use VSD we take the top N tracks of every users based on the affinity they have with their known songs, in the case where we would deal with Spotify data we would use the affinity inference method introduced in Chapter 4 to establish these top N songs. N is a value that has to be defined manually, we established that if the first 25 songs that are recommended are similar to the top 25 tracks of the users then it's a pretty good recommendation.

5.1.2 Dealing with multiple users

When dealing with multiple users, we measure the similarity score of recommendations for every users and we average these scores and that gives us a final score.

5.2 MPR, NDCG and Recall

We spent a lot of time implementing the different metrics found in the different papers we used such as the Mean Percentage Ranking (MPR) [Johnson \(2014\)](#), the [Normalized Discounted Cumulative Gain \(NDCG\)](#) and the Recall [Liang et al. \(2018\)](#) so we could compare the different collaborative-filtering methods. We succeed making the MPR work with the VAE introduced in Chapter 9 but we doubt that the NDCG and Recall work for the LMF since our models yield results that differ a lot, so they will not be discussed further more in this report. However it is a great tool to have for different projects and to select the best VAE model. As a result, we will only use MPR to compare the collaborative models together.

Unlike our Vector Set Divergence (VSD) metric introduced in Section 5.1, they all take into account the order of the recommendation but they do not take into account the features of the items they work on. As a result, the VSD evolution over epoch will lead to a noisy curve since the objective of the collaborative models isn't to find the most similar items (discussed deeper in the sections 8.5 and 9.5).

- **VSD:** The higher, the better
- **MPR:** The lower, the better
- **NDCG:** The higher, the better
- **Recall:** The higher, the better

6. Content-based filtering using Nearest Neighbors

The first paradigm we decided to explore was the content-based filtering one. It provides personalized recommendations based on the intrinsic features of the items we want to recommend. In our case, we have 14 available characteristics for each songs as introduced in [2.1.1](#). We are then able to create a listening profile for each user, by taking their top 150 liked songs.

One key advantage of content-based filtering is its independence from user interactions or the preferences of other users. It doesn't rely on collaborative data or social connections; instead, it's all about understanding the intrinsic characteristics of the items and aligning them with the user's known preferences.

However, content-based filtering also comes with its limitations. For example, due to its tendency to strengthen existing preferences based on identified patterns, it might struggle to introduce users to entirely new genres or unexpected choices. Additionally, if the system only has a few information about the items or the user, it might not generate accurate recommendations

To explore this paradigm we decided to implement the nearest neighbors (NN) algorithm. It is simply based on identifying similarities between data points according to a certain metric, it could be the euclidian distance, the cosine similarity... It enables the algorithm to further make predictions or classify objects. The underlying principle of Nearest Neighbor methods is the idea that similar data points in a feature space tend to exhibit similar characteristics or behaviors. One of the key advantages of NN methods is their simplicity and intuitiveness. Rather than relying on complex mathematical models, NN methods operate by measuring the proximity of instances in a dataset.

6.1 Expected results and thoughts

Since the Nearest Neighbors model picks the closest tracks to the ones we make the recommendation from, we expect that the VSD introduced in [Section 5.1](#) will be close to 0 since the songs have to be similar to each others to be placed 'closely' in the feature space. But we tend to think that it could be lacking generalization capabilities, in particular when we perform recommendation for users the model doesn't know about (fitted without the songs of this user).

However if the feature space lacks of information, for instance if we miss the language of a song, and a user does not listen to music in that language. By adding more and more tracks, we are flooding the space with more data points that might be close to the ones we make the recommendation from. However this song wouldn't have been recommended if a language feature was present. So we would decrease the recommendation quality perceived by the user.

6.2 Key difference with collaborative-filtering methods

A key difference between the Nearest Neighbors and the collaborative-filtering based models is that we can fit the Nearest Neighbors with data that isn't directly related to the users. This data comes from Spotify and might not be present in any user's playlists, liked or top tracks. Despite this, we will still provide a final score in our results without this data addition, because this way, we can compare with more correctness (because they have the same amount of information in the input) to the collaborative systems. And also, as we will discuss further in Chapter 11, the time required for inference increase considerably along with the number of items, so flooding the space with even more songs will lead to a really slow model. It would be smarter to first filter the necessary songs we want to be recommended before fitting the model, leading to a deeper data processing and analysis which may require a human intervention.

6.3 Data processing

6.3.1 Loading the data

We load the wanted artificial datasets by specifying its size as introduced in 2.2.1. If we would want to work with the Spotify data we defined a method to load the dataset to which we can pass multiple arguments to define the affinity inference process defined in Chapter 4, we can also choose to include enriched data or not.

6.3.2 Dropping the duplicates {"username", "id"}

Since a song can be in multiple playlists of the same user, as well as in their top/liked tracks we have to remove the duplicates which have the same keys {"username", "id"}. By default pandas keep the first occurrence so we first sort the dataset on the keys {"username", "affinity"} to keep the occurrence which has the best affinity for that item.

6.3.3 Scaling the numerical features

To have a more reliable VSD score (cf. Section 5.1) we fit a standard scaler using sklearn and transform the numerical features of our dataset. To answer our scientific question this operation might be unnecessary since we are doing it for every model so relatively it would have no difference.

6.3.4 Splitting the user sets

To efficiently evaluate our model we partitioned our dataframe based on the users in three sub-dataset: Train set, Validation set and Test set. We first retrieve the set of unique users of our dataset. Then, we take 80% of the users we have as the development set and the 20% left as the test set, then we split the development set into a train set and validation with respectively 70% and 30%. We used this method to match the VAE's one, in fact this splitting framework will work more like a data ablation study.

Once these datasets are built we simply populate the tracks of these users into the three sub-datasets (train, validation, test). So for the train set the songs of Larry, Daniel, Virginia would

Dataset	Users included
Train set	Larry Daniel Virginia
Validation set	Sean Meagan
Test set	Timothy Todd

Table 6.1: The result of users split from our original dataset

be included (stacked into one dataframe). For the validation set the ones from Sean and Meagan, and for the test set it would be the tracks of Timothy and Todd.

We choose this method of splitting and model selection because it also corresponds to how it is done with our collaborative filtering models from [Liang et al. \(2018\)](#) and [Johnson \(2014\)](#) allowing for a more accurate comparison of the performances. To correctly compare them we perform a generalization analysis by performing a data ablation process, we fit the Nearest Neighbor with the train set containing only a part of the users and calculate the VSD. Then we recommend songs for the users in the validation set and calculate the VSD score, afterward we do the same with the users test set to conclude with a VSD score corresponding to the final generalization capability of the model.

6.4 Model selection

6.4.1 Grid Search

We set a grid of hyperparameters to test which are:

```
param_grid = {
    "n_neighbors": [2, 3, 4, 5, 10],
    "metric": ["cosine", "euclidean"],
    "radius": [0, 0.01, 0.05, 0.1, 0.5],
}
```

Then we test every combination of these hyper-parameters (50 combinations) by fitting the Nearest Neighbors model with the train data and measure the VSD [5.1](#) with our established ground truth [5.1.1](#), this gives a training score. Still fitted with only the training data, we recommend songs to the users that are in the validation set (the model doesn't know about the songs of these people since it was fitted on the training users set) and measure the VSD on this recommendation, this gives us a validation score and shows how well our model generalise to unseen users which have other tastes and preferences.

As we said this process is a data ablation study to test the generalization capability of the model. We refer to it as "train, validation, test" to match how the data are decomposed in the VAE. This allows us to compare the models with less biases and more correctness.

6.4.2 Test

Once we found the best model which is still fitted with the training set, we use the test set to have a final score for this model which is still fitted only with the train data.

6.5 Inference

To get the recommendation, we simply sample the N top tracks of the users, which is in fact our ground truth as in 5.1.1 (picking the ones with the highest affinity value). Then we get their closest k neighbours using our model, with k determined by the Grid Search (`n_neighbors`). Once we retrieved the k closest tracks of each tracks of the ground truth, we end up with $N \cdot k$ tracks recommended which have a distance property to the song it was recommended from. We sort them by the distances they have with their original track and drop the duplicates. That gives us the recommended tracks.

Recommendation space



Figure 6.1: nn'viz.png - An example of a space with the tracks of a user (blue) and the recommended tracks (red). The space has been reduced to 3 dimensions with PCA to be able to produce this visualization.

7. Collaborative filtering with implicit feedback

Collaborative filtering with implicit feedback has gained prominence in online recommendation systems due to its practicality. Unlike explicit feedback, which relies on users providing specific ratings or preferences, implicit feedback is inherently more abundant as it is derived from user interactions, such as clicks, views, or purchase history. This wealth of implicit data is readily available in large-scale online platforms, making it more convenient and cost-effective to collect. Additionally, implicit feedback addresses the sparsity issue often encountered with explicit feedback, where users may not consistently rate or provide feedback on items.

Our approach to collaborative filtering with implicit feedback is built upon the implementation introduced in [Johnson \(2014\)](#). In the following sections, we will delve into an explanation of this implementation, providing detailed insights into its workings. In this paper, they explore a new method for collaborative filtering with implicit feedback data that's based in part on the traditional matrix factorization strategy. We will explain the theoretical foundations of the model and how to train it.

We choose the Multi Variational Model by [Liang et al. \(2018\)](#) and the Logistic Matrix Factorization by [Johnson \(2014\)](#) because both were very well described by platforms such as Google, Netflix and Spotify. Also they are based on two different mechanism so we could cover a wider variety of collaborative models.

7.1 Notations

For the rest of our collaborative filtering approaches, we will assume that we have:

- $U = (u_1, \dots, u_n)$ a group of n users
- $I = (i_1, \dots, i_m)$ a group of m items
- $R = (r_{u,i})_{n \times m}$ a user-item observation matrix where $r_{u,i} \in \mathbb{R}^+$ represents the number of times user u interacted with item i .
- $r_u = [r_{u,1} \quad \dots \quad r_{u,m}]$ a vector containing the affinity for each items from user u .

7.2 Data processing

For all the collaborative filtering models, our data processing is almost the same.

7.2.1 Loading the data, dropping the duplicates and scaler

We perform the same loading process as for the Nearest Neighbor model outlined in [6.3.1](#), with the same affinity inference parameters as in [Section 4.6](#). We also perform the same pre-processing operations such as dropping the duplicated on the {"username", "id"} keys [6.3.2](#) and scaling our numerical features with a standard scaler [6.3.3](#).

7.2.2 Keeping the tracks that appears at least N times

What was done in [Liang et al. \(2018\)](#) that improves a lot our performances in term of the NDCG, MPR and Recall metrics 5.2, is to keep only the tracks that appears N times in our dataset. We found that $N = 2$ is a great value since we do not have a big dataset. For that we implemented a function to which we pass our DataFrame, the column name we work on and the number of minimum occurrence every entry must have: `preprocessing.filter_rows_by_min_number_of_occurence(df, "id", 2)`.

Recall that we dropped the duplicated keys on {"username", "id"} which doesn't drop the duplicated tracks in our dataset, user A could have item i as well as user B . Leading to the fact that `filter_rows_by_min_number_of_occurence` will only keep the tracks shared by at least 2 users.

7.2.3 Splitting the user set

We split our user set the same way we did for the Nearest Neighbors model in 6.3.4 and with the same parameters. But note that we build the final dataset differently. In fact, instead of creating three sub datasets (train, validation, test) which contains all the tracks of users, we use the same strategy as the VAE model from [Liang et al. \(2018\)](#). They subdivide the validation set and the test set into 2 sub-dataframe, ending up with 5 dataframes (train, validation-train, validation-test, test-train, test-test) instead of three with our Nearest Neighbors. We used the same naming convention these sub-dataframes as the reference implementation we could find for the VAE [Liang \(2018\)](#) [Seo \(2021\)](#).

Train dataset

We populate the train dataset with all the tracks from the users contained in the user train set.

Validation dataset

From the user validation set we sample half the songs of every users in the validation-train dataset and half in the validation-test dataset. If a user has less than a threshold t that we manually set, then all the user's songs will be placed in the validation-train dataset, we used $t = 2$.

Test dataset

The same thing as for the validation set is done to build the test-train and the test-test datasets which are build from the user test set.

7.2.4 Building the interaction matrix

To train the collaborative systems the input has to be in a interaction matrix format with $|U|$ rows which represent the user and $|I|$ columns representing the items. The entries of this matrix represent the affinity that user u has with an item i .

In the context of implicit feedback, we only know when a user interacted with an item. **We do not know if they liked it or disliked it.** However we can make assumption as we did in Chapter 4 to infer an affinity between an user and an item. In this context the convention is to set the entries to 0 when a user u didn't interact with an item i , usually we end up with a really sparse matrix.

We can calculate the matrix sparsity from 0 to 1 with the following formula:

$$S := \frac{|D|}{|U| \cdot |I|}$$

D : The original data set

U : The set of unique users in D

I : The set of unique items in D

We implemented a Python class which turns a flat dataset to an interaction matrix, so for instance if we have the following dataset:

username	id	affinity
user_1	item_1	1.0
user_1	item_2	0.3
user_1	item_3	0.4
user_2	item_1	0.5
user_3	item_2	1.0
user_3	item_3	0.6

Table 7.1: An example raw dataset without duplicates

it will turn into the following matrix:

user index (username) / item index (id)	0 (item_1)	1 (item_2)	2 (item_3)
0 (user_1)	1.0	0.3	0.4
1 (user_2)	0.5	0	0
2 (user_3)	0	1.0	0.6

Table 7.2: The matrix R generated by the example dataset

This class also helps us to map the matrix indices with the usernames and item Ids which helps us to perform mapping while we perform our recommendation process

We build these interaction matrices for each of the sub-datasets we build, train, validation-train, validation-test, test-train and test-test. Ending up with 5 individual matrices.

7.2.5 Scaling the matrix entries

As proposed by [Johnson \(2014\)](#), we compute a factor α which scales the entries of the matrix to balance the importance to zero/non-zero entries of the matrix. Once computed R is simply scaled by α as followed: $R := \alpha R$. We used what they proposed using the the following formula:

$$\alpha(R) := \frac{|R| - |R_{\neq 0}|}{\sum R}$$

$|R|$: The number of entries of R
 $|R_{\neq 0}|$: The number of non zero entries of R
 $\sum R$: The sum of the entries of R

To handle the different train, validation and test subsets we compute α on the original dataset before performing the splitting strategy and we apply the α scale to the train, validation and test subsets afterward.

After all these pre-processing steps, the matrices datasets are ready to be processed by our collaborative-based models.

8. Logistic matrix factorization

The model adopts a similar strategy as outlined in [Hu et al. \(2008\)](#). However, instead of minimizing a weighted RMSE, it uses a probabilistic approach introduced by [Mnih and Salakhutdinov \(2007\)](#), where the assumption is made that the entries follow a Gaussian distribution. In this context, the authors present a novel probabilistic framework tailored for implicit scenarios. Within this framework, the probability of a user selecting an item is modeled using a logistic function. The model achieves factorization of matrix R into two lower-dimensional matrices, $X_{n \times f}$ and $Y_{m \times f}$, where f represents the number of latent factors.

To implement the code in we based our implementation on the authors one from his GitHub repository: <https://github.com/MrChrisJohnson/logistic-mf>. We converted everything to be able to use PyTorch with the GPU to make the learning phase faster.

- The rows of X are latent factor vectors representing a user's taste
- The columns of Y^T are latent factor vectors representing an item's style, genre, ...

8.1 General probabilistic framework

The general probabilistic matrix factorization framework is based on Bayesian learning where the aim is to find a posterior distribution of the model parameters by using the Bayes rule:

$$p(\theta|X, \alpha) = \frac{p(X|\theta, \alpha)p(\theta|\alpha)}{P(X|\alpha)} \propto p(X|\theta, \alpha)p(\theta|\alpha)$$

with X the dataset, θ the parameter set of the distribution, α the hyperparameter of the distribution, $p(\theta|X, \alpha)$ the posterior, $p(X|\theta, \alpha)$ the likelihood and $p(\theta|\alpha)$ the prior.

8.2 Our framework

In our case, we have:

$$\begin{aligned} X &= R \\ \theta &= X, Y \\ \alpha &= \sigma_u^2, \sigma_i^2 \end{aligned}$$

So we have the following posterior:

$$p(X, Y|R, \sigma_u^2, \sigma_i^2) = p(R|X, Y, \sigma_u^2, \sigma_i^2)p(X, Y|\sigma_u^2, \sigma_i^2)$$

As X and Y are independent:

$$p(X, Y|R, \sigma_u^2, \sigma_i^2) = p(R|X, Y, \sigma_u^2, \sigma_i^2)p(X|\sigma_u^2)p(Y|\sigma_i^2)$$

Now, we need to find each term on the right hand side of the equation.

Let $l_{u,i}$ denote the event that user u has interacted with item i . Then they consider that the probability of this event is distributed according to a logistic function:

$$p(l_{u,i}|x_u, y_i, \beta_u, \beta_i) = \frac{\exp(x_u y_i^T + \beta_u + \beta_i)}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)}$$

With β_i and β_u introduced as biased meant to account for variation in behavior across both users and items.

The formulation introduces a confidence measure, $c = \alpha r_{u,i}$, based on a tuning parameter, α , where non-zero entries in the observation matrix R serve as positive observations, and zero entries as negative observations, with α allowing for the adjustment of weights between positive and negative observations.

By considering zero-mean spherical Gaussian priors on user and item latent factor vectors and that all entries of R are independent, the posterior can be written as:

$$p(X, Y, \beta | R) = \prod_{u,i} p(l_{u,i}|x_u, y_i, \beta_u, \beta_i)^{\alpha r_{u,i}} (1 - p(l_{u,i}|x_u, y_i, \beta_u, \beta_i)) \prod_u \mathcal{N}(x_u | 0, \sigma_u^2) \prod_i \mathcal{N}(y_i | 0, \sigma_i^2)$$

If we take the log of the posterior we have:

$$\begin{aligned} & \sum_{u,i} \alpha r_{u,i} \ln(p(l_{u,i}|x_u, y_i, \beta_u, \beta_i)) + \ln(1 - p(l_{u,i}|x_u, y_i, \beta_u, \beta_i)) + \sum_u \ln(\mathcal{N}(x_u | 0, \sigma_u^2)) + \sum_i \ln(\mathcal{N}(y_i | 0, \sigma_i^2)) \\ \text{Let } z &= \sum_u \ln(\mathcal{N}(x_u | 0, \sigma_u^2)) + \sum_i \ln(\mathcal{N}(y_i | 0, \sigma_i^2)) \\ &= \sum_{u,i} \alpha r_{u,i} \ln\left(\frac{\exp(x_u y_i^T + \beta_u + \beta_i)}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)}\right) + \ln\left(\frac{1 + \exp(x_u y_i^T + \beta_u + \beta_i)}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)} - \frac{\exp(x_u y_i^T + \beta_u + \beta_i)}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)}\right) + z \\ &= \sum_{u,i} \alpha r_{u,i} \ln\left(\frac{\exp(x_u y_i^T + \beta_u + \beta_i)}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)}\right) + \ln\left(\frac{1}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)}\right) + z \\ &= \sum_{u,i} \alpha r_{u,i} (x_u y_i^T + \beta_u + \beta_i) - \alpha r_{u,i} \ln(1 + \exp(x_u y_i^T + \beta_u + \beta_i)) - \ln(1 + \exp(x_u y_i^T + \beta_u + \beta_i)) + z \\ \text{Let } w &= x_u y_i^T + \beta_u + \beta_i \\ &= \sum_{u,i} \alpha r_{u,i} w - (1 + \alpha r_{u,i}) \ln(1 + \exp(w)) - \frac{1}{2\sigma_u^2} \sum_u x_u^T x_u - \frac{1}{2\sigma_i^2} \sum_i y_i^T y_i \end{aligned}$$

We finally have:

$$\ln(p(X, Y, \beta | R)) = \sum_{u,i} \alpha r_{u,i} (x_u y_i^T + \beta_u + \beta_i) - (1 + \alpha r_{u,i}) \ln(1 + \exp(x_u y_i^T + \beta_u + \beta_i)) - \frac{1}{2\sigma_u^2} \|x_u\|^2 - \frac{1}{2\sigma_i^2} \|y_i\|^2$$

8.3 Learning the parameters

The goal now, is to learn X, Y, β that maximizes the log posterior.

$$\arg \max_{X,Y,\beta} \ln (p(X,Y,\beta|R))$$

This can be done with an alternating gradient ascent. Where at each iteration, we first fix the vectors X and β and take a step towards the gradient of the vectors Y and β . Next we do it the other way around. Let's compute these gradients:

Firstly,

$$\begin{aligned} \text{Using the chain rule for: } \frac{\partial}{\partial \beta_u} \ln (1 + \exp(x_u y_i^T + \beta_u + \beta_i)) &= \frac{\partial \ln(a)}{\partial a} \frac{\partial a}{\partial \beta_u} \text{ with } a = 1 + \exp(x_u y_i^T + \beta_u + \beta_i) \\ \frac{\partial}{\partial \beta_u} \ln (p(X,Y,\beta|R)) &= \frac{\partial}{\partial \beta_u} \sum_{u,i} \alpha r_{u,i} (x_u y_i^T + \beta_u + \beta_i) - (1 + \alpha r_{u,i}) \ln (1 + \exp(x_u y_i^T + \beta_u + \beta_i)) - \frac{1}{2\sigma_u^2} \|x_u\|^2 - \frac{1}{2\sigma_i^2} \|y_i\|^2 \\ &= \sum_i \alpha r_{u,i} - (1 + \alpha r_{u,i}) \frac{\exp(x_u y_i^T + \beta_u + \beta_i)}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)} \end{aligned}$$

Secondly, with the same method we have

$$\frac{\partial}{\partial \beta_i} \ln (p(X,Y,\beta|R)) = \sum_u \alpha r_{u,i} - (1 + \alpha r_{u,i}) \frac{\exp(x_u y_i^T + \beta_u + \beta_i)}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)}$$

In the same way, we get:

$$\frac{\partial}{\partial x_u} \ln (p(X,Y,\beta|R)) = \sum_i \alpha r_{u,i} y_i - (1 + \alpha r_{u,i}) y_i \frac{\exp(x_u y_i^T + \beta_u + \beta_i)}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)} - \frac{x_u}{\sigma_u^2}$$

and finally,

$$\frac{\partial}{\partial y_i} \ln (p(X,Y,\beta|R)) = \sum_u \alpha r_{u,i} x_u - (1 + \alpha r_{u,i}) x_u \frac{\exp(x_u y_i^T + \beta_u + \beta_i)}{1 + \exp(x_u y_i^T + \beta_u + \beta_i)} - \frac{y_i}{\sigma_i^2}$$

They address the linear time complexity issue in large datasets by reducing negative samples and adjusting parameters for faster computation. Additionally, they explore adaptive gradient step sizes using AdaGrad, potentially reducing the number of iterations needed for convergence. The AdaGrad update for the variable x_u at iteration t is:

$$x_u^t = x_u^{t-1} + \frac{\gamma g_u^{t-1}}{\sqrt{\sum_{t'=1}^{t-1} (g_u^{t'})^2}}$$

8.4 Prediction

Once we have determined both X and Y and β that maximizes the log posterior, we are able to predict the interactions for all users and items by simply taking: $X \cdot Y^T + \beta$. That gives the predicted affinity matrix, where every rows represents a user. We can simply argsort these rows and retrieve the indices of the items to build a recommendation list.

8.5 Training process and model selection

Following [Johnson \(2016\)](#) implementation we could effectively train the model with all our generated datasets built in 2.2.1 (and ending up with similar loss curves outlined in Figure 8.1 for every datasets):

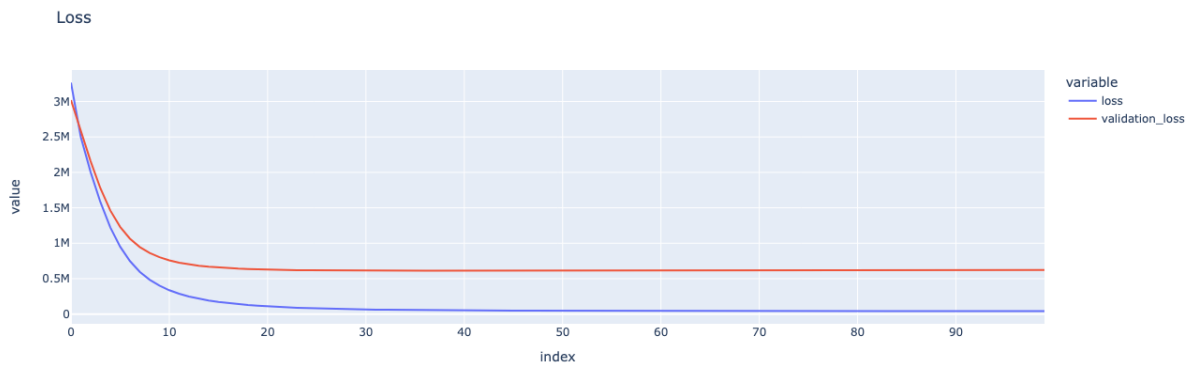


Figure 8.1: lmf128.png - The training loss and validation loss we have with the 128 users dataset

The MPR metric introduced by the LMF paper adopts the expected results, decreasing and converging to 0 as shown in Figure 8.2. Showing that it effectively learns.

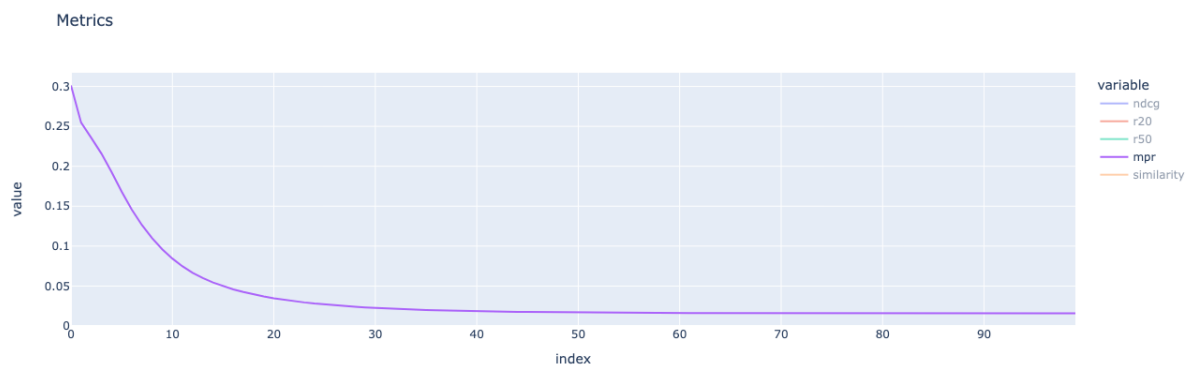


Figure 8.2: lmfmpr.png - The MPR metric

The VSD converges really quickly to it's final value and adopts an noisy signal as outlined in Figure 8.3, it is probably because the loss function does not operate on item's features. The model does not directly learn how to provide recommendation on similar items but based on an interaction matrix. Unlike the MPR or NDCG metrics which operated on the item's rank in the recommendation list, which is what the model is capable to learn.

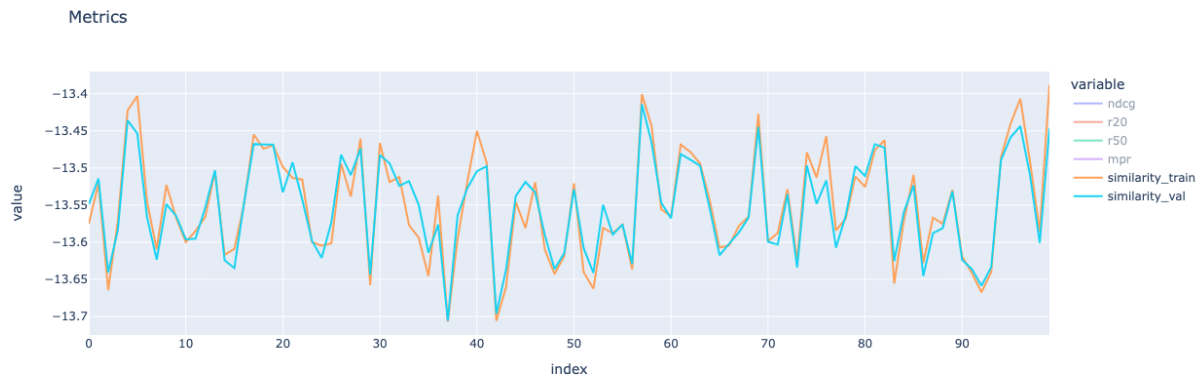


Figure 8.3: lmf vsd.png - The VSD metric

The other metrics, NDCG and Recall, do not really adopt the expected behaviour described by the VAE paper as shown in Figure 8.4. They maybe aren't suited for this model, further analysis can be done to verify the cause, but since it will not help answering our scientific question we will not spend more time on this aspect.

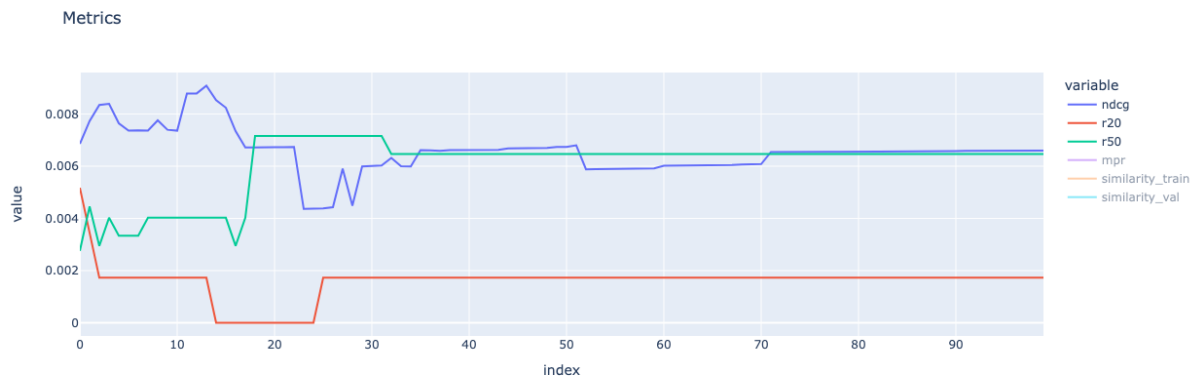


Figure 8.4: lmf metrics.png - The NDCG and Recall metrics

8.6 Latent space visualization

Since this model learns latent representation of users and items we can simply plot the produced latent features into a space and analyse the objects proximity. That's what we implemented to understand the latent space. To produce the plot we can train the model to have $N > 3$ features and use a pair plot or a dimensionality reduction method like PCA or directly train the model with $N \leq 3$ and plot it directly. To produce a more meaningful plot we choose the second option here:

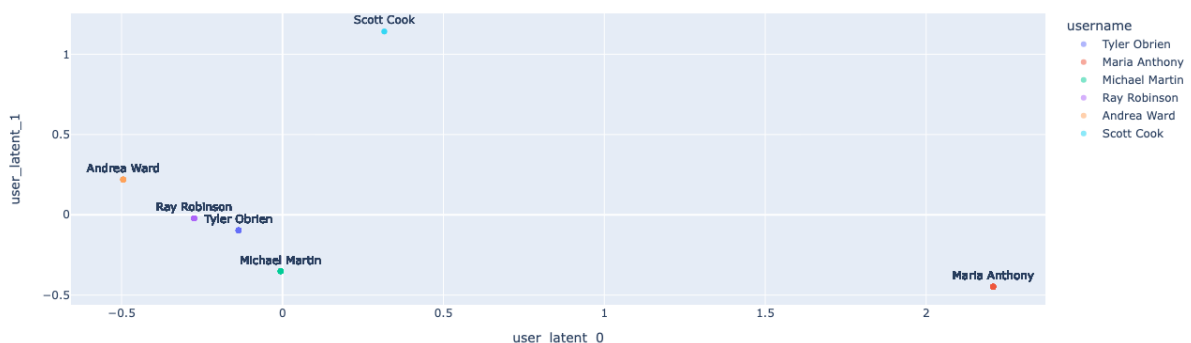


Figure 8.5: lmf viz users.png - The 2D users latent representations

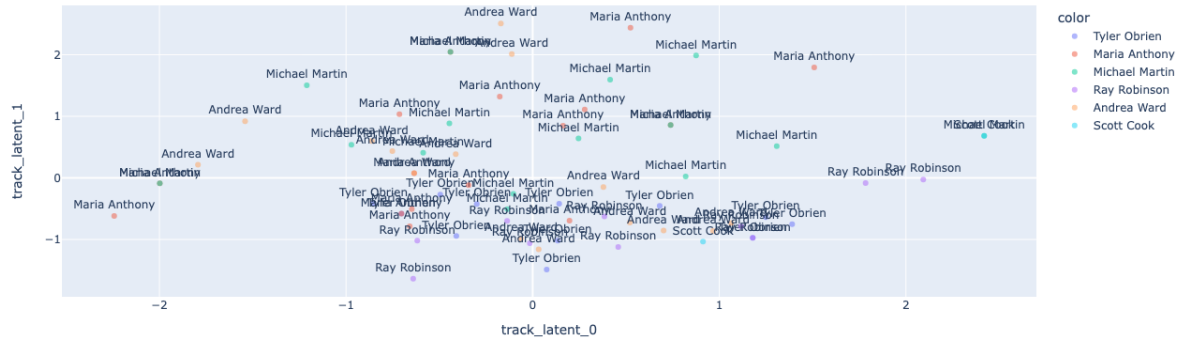


Figure 8.6: lmf viz tracks.png - The users tracks 2D users latent representations

9. Variational Autoencoders

An autoencoder is a neural architecture that consists of an encoder and a decoder. The encoder takes input data and maps it to a lower-dimensional representation (latent space), while the decoder takes this lower-dimensional representation and reconstructs the original input data as closely as possible.

Variational inference is a method used to approximate complex probability distributions and involves transforming the problem of finding the posterior distribution into an optimization problem.

Variational autoencoders serve as probabilistic generative models that incorporate neural networks as components. These neural network elements are commonly denoted as the encoder and decoder for the first and second components, respectively. The initial neural network maps the input variable to a latent space, aligning with the parameters of a variational distribution. This enables the encoder to generate diverse samples originating from the same distribution. Conversely, the decoder functions in reverse, mapping from the latent space to the input space to generate or produce data points.

We will follow [Liang et al. \(2018\)](#) for this implementation and keep their notations.

9.1 The general model

For each user u we sample a K – dimensional latent representation z_u from a Gaussian prior. Then, z_u is transformed via a non-linear function $f_\theta(\cdot) \in \mathbb{R}^I$ which gives $\pi(z_u)$ a probability distribution over I items from which $r_{u,i}$ $i = 1, \dots, m$ is assumed to have been drawn. $f_\theta(\cdot)$ is a multilayer perceptron with parameters θ that can be represented as the concatenation of all weight and bias parameters across all layers:

$$\theta = \{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, \dots, W^{(L)}, b^{(L)}\}$$

where L is the total number of layers in the MLP. The output is then normalized via a softmax function that finally gives the probability vector $\pi(z_u)$.

The softmax function takes as input a vector of real numbers and produces a probability distribution over the elements of the vector. Given an input vector $z = (z_1, z_2, \dots, z_k)$, the softmax function computes the probability p_i of the i -th element as following:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

The generative model that generalizes the latent factor model can be written as:

$$\begin{aligned} z_u &\sim \mathcal{N}(0, I_K) \\ \pi(z_u) &\propto \exp\{f_\theta(z_u)\} \\ r_u &\sim \text{Mult}(N_u, \pi(z_u)) \quad \text{With } N_u = \sum_i r_{u,i} \quad \text{Note that here } r_{u,i} \text{ represents affinity.} \end{aligned}$$

Finally, with the probability mass function (PMF) of a multinomial distribution, we have:

$$\ln p_\theta(r_u|z_u) = \sum_i r_{u,i} \pi_i(z_u)$$

Note that by using the logistic likelihood and setting f_θ to be linear, we have the same equation as in the previous section.

9.2 Variational inference

The goal is to learn the generative model we introduced above:

- $z_u \sim \mathcal{N}(0, I_K)$ the latent variable for each user is sampled from a multivariate normal (Gaussian) distribution, K is the dimensionality of the latent space.
- $\pi(z_u) \propto \exp\{f_\theta(z_u)\}$ $\pi(z_u)$ transforms the latent space representation into a probability distribution via a neural network with parameters θ and normalized with a softmax function.
- $r_u \sim \text{Mult}(N_u, \pi(z_u))$ The observed data r_u for user u is generated from a multinomial distribution. The parameters of this multinomial distribution are determined by $\pi(z_u)$, the probability distribution over the items. The parameter N_u is the sum of the user's u affinity with its items.

and especially, we need to estimate θ to learn this model. To do so, they use variational inference to approximate the true intractable posterior $p(z_u|r_u)$ with an easier variational distribution $q(z_u)$ a fully factorized (diagonal) Gaussian distribution.

$$q(z_u) = \mathcal{N}(\mu_u, \text{diag}(\sigma_u^2))$$

This defines a probability distribution for the variable z_u to be a Gaussian distribution where the variables are uncorrelated, and the covariance matrix is diagonal. This choice simplifies the distribution by assuming independence between the components of z_u .

Now the objective of variational inference, is to optimize $\{\mu_u, \sigma_u^2\}$ such that the Kullback-Leiber divergence is minimized.

$$D_{KL}(q(z_u)||p(z_u|r_u))$$

One of the main problem is that this optimization needs to be done for each data point. Meaning that a recommender system with millions of users and items will be very slow. This is where variational autoencoders come into play as they use a data-dependent function, usually called inference model parametrized by ϕ :

$$g_\phi(r_u) = [\mu_\phi(r_u) \quad \sigma_\phi(r_u)] \in \mathbb{R}^{2K}$$

where $\mu_\phi(r_u)$ and $\sigma_\phi(r_u)$ are two vectors in $\mathbb{R}^K$. We then set the variational distribution to be:

$$q_\phi(z_u|r_u) = \mathcal{N}(\mu_\phi(r_u), \text{diag}\{\sigma_\phi^2(r_u)\})$$

Finally, using the data we know r_u , the inference model outputs the variational parameter ϕ of $q_\phi(z_u|r_u)$. When optimized, this variational distribution approximates the posterior we search. Putting this variational distribution and the generative model together, we have:

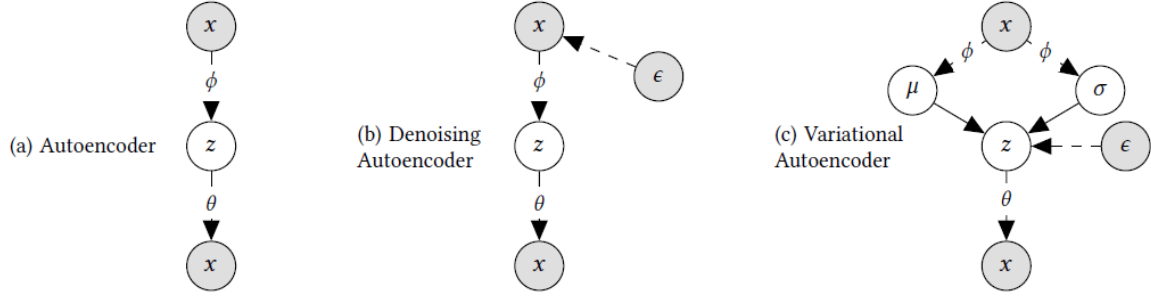


Figure 9.1: taxonomy.png - A taxonomy of autoencoders given in [Liang et al. \(2018\)](#)

9.3 Learning the parameters

To have a deeper understanding of how the learning can be done, we checked [Kingma and Welling \(2022\)](#) which is also mentioned in [Liang et al. \(2018\)](#).

The log marginal likelihood for one user, which measures how well the generative model parameterized by θ explains or generates the observed data, can be expressed as:

$$\ln p_\theta(r_u) = D_{KL}(q_\phi(z_u|r_u)||p_\theta(z_u|x_u)) + \mathcal{L}(\phi, \theta; r_u)$$

With $\mathcal{L}(\phi, \theta; r_u)$ being the evidence lower bound (ELBO) defined as:

$$\mathcal{L}(\phi, \theta; r_u) := \mathbb{E}_{q_\phi(z_u|r_u)} [\ln p_\theta(r_u, z_u) - \ln q_\phi(z_u|r_u)]$$

and as:

$$\begin{aligned} \mathbb{E}_{q_\phi(z_u|r_u)} [\ln p_\theta(r_u, z_u) - \ln q_\phi(z_u|r_u)] &= \mathbb{E}_{q_\phi(z_u|r_u)} [\ln p_\theta(r_u|z_u) + \ln p_\theta(z_u) - \ln q_\phi(z_u|r_u)] \\ &= \mathbb{E}_{q_\phi(z_u|r_u)} [\ln p_\theta(r_u|z_u)] + \mathbb{E}_{q_\phi(z_u|r_u)} [\ln p_\theta(z_u) - \ln q_\phi(z_u|r_u)] \end{aligned}$$

Let's only have a look at:

$$\begin{aligned} \mathbb{E}_{q_\phi(z_u|r_u)} [\ln p_\theta(z_u) - \ln q_\phi(z_u|r_u)] &= \mathbb{E}_{q_\phi(z_u|r_u)} \left[\ln \frac{p_\theta(z_u)}{q_\phi(z_u|r_u)} \right] \\ &= \int q_\phi(z_u|r_u) \ln \frac{p_\theta(z_u)}{q_\phi(z_u|r_u)} dz_u \\ &= - \int q_\phi(z_u|r_u) \ln \frac{q_\phi(z_u|r_u)}{p_\theta(z_u)} dz_u \\ &= -D_{KL}(q_\phi(z_u|r_u)||p_\theta(z_u)) \end{aligned}$$

Which finally gives us:

$$\ln p_\theta(r_u) \geq \mathcal{L}(\phi, \theta; r_u) = \mathbb{E}_{q_\phi(z_u|r_u)} [\ln p_\theta(r_u|z_u)] - D_{KL}(q_\phi(z_u|r_u) || p_\theta(z_u))$$

The lower bound we seek to maximize for the user u and that's why it takes its name. Note that this function depends on both ϕ and θ parameters. But the main problem is to take the gradient with respect to ϕ , to solve this issue, [Kingma and Welling \(2022\)](#) introduce a reparametrization trick. We will not go into the details, but they sample $\epsilon \sim \mathcal{N}(0, I_K)$ and reparametrize $z_u = \mu_\phi(r_u) + \epsilon \odot \sigma_\phi(r_u)$ to be able to back-propagate the gradient with respect to ϕ .

The whole training part can be written as the following pseudo-code:

Algorithm 1 VAE-SGD Training collaborative filtering VAE with stochastic gradient descent from [Liang et al. \(2018\)](#).

Require: Affinity matrix $\mathbf{X} \in \mathbb{R}^{U \times I}$

```

1: Randomly initialize  $\theta, \phi$ 
2: while not converged do
3:   Sample a batch of users  $U$ 
4:   for all  $u \in U$  do
5:     Sample  $\epsilon \sim \mathcal{N}(0, \mathbf{I}_K)$  and compute  $z_u$  via reparameterization trick
6:     Compute noisy gradient  $\nabla_\theta \mathcal{L}$  and  $\nabla_\phi \mathcal{L}$  with  $z_u$ 
7:   end for
8:   Average noisy gradients from batch
9:   Update  $\theta$  and  $\phi$  by taking stochastic gradient steps
10: end while
11: return  $\theta, \phi$ 

```

Finally, after defining formally this objective, they consider some trade-off mainly to increase the performances. In fact, as in the problem of image inpainting in computer vision, they look at the ELBO as a reconstruction error term and a regularization term with a parameter β to control its strength. This alternate ELBO is defined as:

$$\mathcal{L}_\beta(\phi, \theta; r_u) := \mathbb{E}_{q_\phi(z_u|r_u)} [\ln p_\theta(r_u|z_u)] - \beta \cdot D_{KL}(q_\phi(z_u|r_u) || p_\theta(z_u))$$

But how to select the best β parameter ?

The authors propose a simple heuristic during the training: set β as a free regularization parameter and start with $\beta = 0$ and gradually increase it to 1. Annealing the KL term slowly over multiple updates. The best β is determined based on the peak validation metric. Once the best β is found, we retrain the model and we stop increasing β after reaching the annealing schedule.

The main idea of annealing here, is to allow the model to initially focus on the reconstruction error and then gradually take the regularization term into account. This approach can improve the overall performance of the VAE.

9.4 Prediction

Once the generative model is trained, we proceed as following to make the predictions. Given the affinity vector of user u r_u , we firstly build the latent representation z_u by taking the mean of the variational distribution:

$$z_u = \mu_\phi(r_u)$$

And then, based on the un-normalized predicted multinomial probability $f_\theta(z)$ we rank the items.

Note that one key advantage compared to the matrix factorization we discussed previously is that, when given a affinity vector of a user not seen in the training, we don't need to perform any kind of optimization to obtain the latent representation of this user.

9.5 Training process and model selection

For the LMF model we used a Stochastic Gradient Descent mode, we decided to do the same for the VAE to compare their training time in the same conditions and also because we do not have a large amount of user. However the implementation allows us to easily increase the number of user per batch by changing a variable `batch_size` in the `recommender.vae.ipynb` notebook.

With our implementation based on [Seo \(2021\)](#) and the author's one [Liang \(2018\)](#) we could effectively train our model:

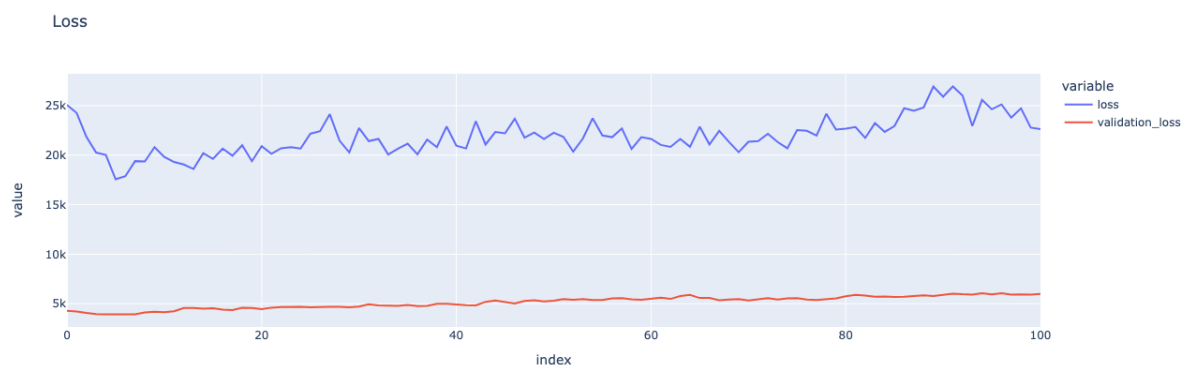


Figure 9.2: train`vae`128.png - The training loss and validation loss we have with the 128 users dataset

Outlined in Figure 9.4 and 9.3 are our evaluation metrics over time for the 128 users dataset, we can see the MPR decreasing, the NDCG/Recall increasing which is what we expect. Since the MPR seems to adopt the expected behavior for both the LMF and VAE we will use that metric to compare the results of the collaborative systems in the Chapter 11. However, we have to be careful with the MPR because it converges to magnitude higher value than with the LMF, we might need to make some adjustment to successfully port the MPR to the VAE.

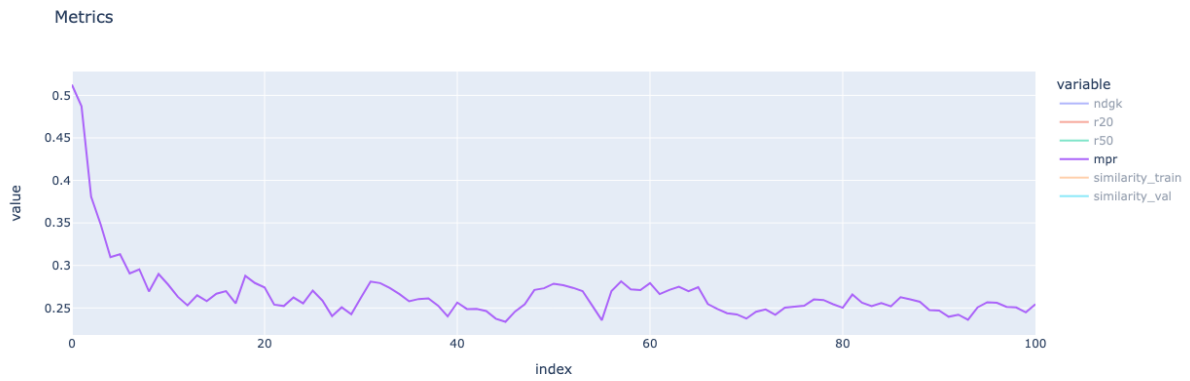


Figure 9.3: vae`mpr.png - MPR



Figure 9.4: vae`metrics.png - NDCG and Recall20, Recall50

Just like for the LMF, the VSD metric converges really quickly to its final value and oscillates around a central value, ending up with a more noisy VSD evolution. We gave an explanation to the phenomenon in the previous Section 8.5.

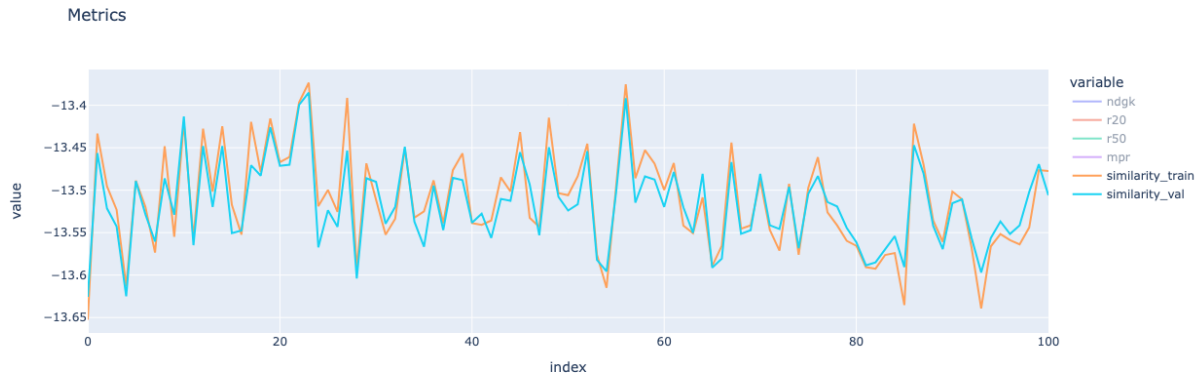


Figure 9.5: vae`vsd.png - VSD

9.6 Latent distribution visualization

Instead of representing a user by a single point in a space like LMF does, the VAE represents them as multivariate distributions. We could use the same visualization method as for LMF and plot a point where the user's mean is but it wouldn't provide the whole information. For that purpose, we developed a tool to visualize the latent distributions, we simply plot the users distributions as a heatmap of every pair of dimensions. Allowing us to better understand what is going on under the hood of this model.

The same visualisation could be done with the tracks but would be harder to interpret. For that reason we only focused on providing the user proximity plot.

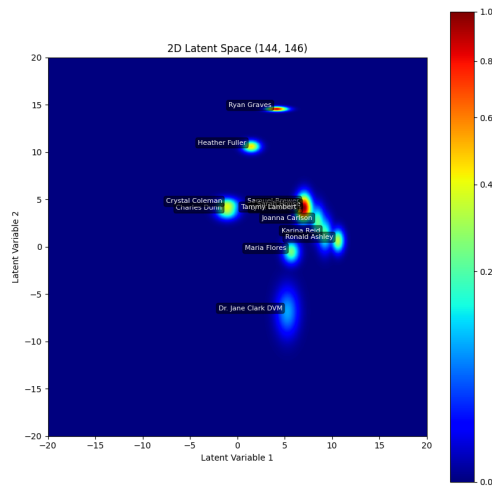


Figure 9.6: vae'viz'1.png - Dimension 144 against 146

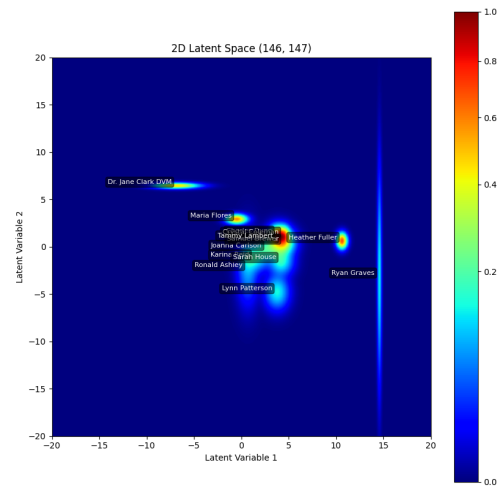


Figure 9.8: vae'viz'3.png - Dimension 146 against 147

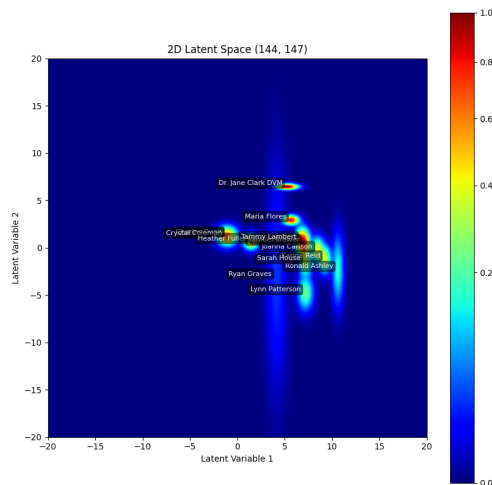


Figure 9.7: vae'viz'2.png - Dimension 144 against 147

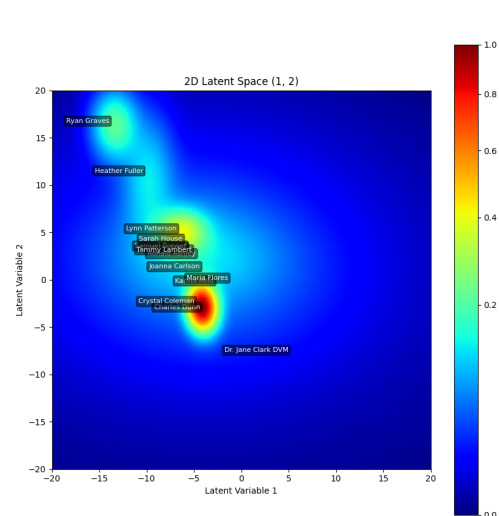


Figure 9.9: vae'viz'4.png - Dimension 1 against 2

10. Ethical discussion

10.1 Privacy, anonymization method and ethical discussion on our project

As discussed in [2.1.3](#), we cannot use people information to train our models. We strictly respected this rule and saw it as common challenge that we will have to deal with in our career since the data privacy and AI are topics rising with importance in our society.

Even though there are no more personally identifiable information in our datasets, it was already shown in 2006 when Netflix launched a competition that it is not sufficient to preserve the privacy. They shared a data of movie ratings, the date the rating was done and a unique ID to describe the user. The goal of the competition was to find a better recommendation system than theirs. Only a couple of weeks later it was shown by [Narayanan and Shmatikov \(2008\)](#) that it was possible to connect many of the IDs to real people, by cross-referencing the publicly available IMDB movie ratings.

Not to forget that these systems collect an enormous amount of data about people, mainly to build these recommender systems and to always improve them. While this data helps in personalizing our experience, it can also be a breach of privacy. How much do these platforms know about us? And who else gets access to this information? For instance, we can recall the [Facebook-Cambridge Analytica data scandal](#) where the data of million of users were use without their consent.

In the end, to keep the scope of the project and because it's not the main goal of this course, we decided not to implement any privacy preserving mechanism. We will simply not release the datasets and remove all possible identifier in the code before publishing it on our GitHub. Of course, for the grading of this project we have to hand them in.

10.2 Ethical discussion on recommender systems

Recommender systems, like the ones used by YouTube, Spotify, TikTok, Instagram and other entertainment platforms are designed to suggest content that we might like. Sounds simple, right? But there's a lot more going on under the hood, these models are the ones that are the closest to us, human, they are built to represent what we like and understand who we are in a very detailed manner.

First, these algorithms learn from our behavior, what we like, share, or spend time watching/listening. They use our data to feed us with more of similar content. Initially, it sounds great because it helps us find items we're interested in without much effort, but it might introduce a lot of ethic concerns.

One big issue is the creation of "filter bubbles.", this phenomenon tends to appear when the systems keeps showing us only what it thinks we want to see. It sounds okay at first, but it can narrow our worldview, limiting our exposure to different perspectives. Consider the scenario where you are only presented with information that are fully aligned to your existing beliefs. For some individuals, it could lead them to extreme way of thinking about different subjects such as conspiracy theories for instance.

We can also talk about the impact on mental health, especially for younger users. Platforms like Instagram and TikTok are known for their addictive nature. They're designed to keep us engaged for as long as possible. This can lead to excessive use, which could be really harmful for mentally vulnerable people. We can see this aspect mixed with "filter bubbles" which can be a really dangerous cocktail to keep in mind when creating such models.

Lastly, fake news and content manipulation are big challenges for the platforms. These systems might promote sensational or misleading content because it tends to get more engagement, affecting public opinion on important matters such as political subjects.

11. Results and discussions

To obtain our results we trained both of the collaborative models on 100 epochs.

11.1 Results

N users	Model	VSD Train	VSD Val.	VSD Test	σ	Train time	MPR	NDCG 100	Recall 50
8	VAE	-13.17	-13.34	-13.46	0.12	27.65s	0.42	0.101	0.45
	LMF	-12.92	-12.95	-13.02	0.04	15.50s	0.44	0.423	0.41
	NN	-12.69	-13.03	-13.09	0.17	1.88s	-	-	-
	NN Full	-13.39	-13.31	-13.41	0.04	4.93s	-	-	-
16	VAE	-13.20	-13.28	-13.29	0.04	27.83s	0.34	0.038	0.33
	LMF	-13.04	-13.33	-13.34	0.14	20.58s	0.35	0.206	0.18
	NN	-13.24	-13.07	-13.06	0.08	7.55s	-	-	-
	NN Full	-13.47	-13.41	-13.41	0.03	21.09s	-	-	-
32	VAE	-13.24	-13.27	-13.34	0.04	110.67s	0.18	0.148	0.8
	LMF	-13.28	-13.38	-13.38	0.05	48.84s	0.26	0.021	0.024
	NN	-13.40	-13.29	-13.36	0.05	49.38s	-	-	-
	NN Full	-13.49	-13.53	-13.56	0.03	128.91s	-	-	-
64	VAE	-13.34	-13.40	-13.42	0.03	232.91s	0.39	0.093	0.440
	LMF	-13.40	-13.48	-13.47	0.04	111.70s	0.34	0.014	0.005
	NN	-13.74	-13.56	-13.59	0.08	157.30s	-	-	-
	NN Full	-13.59	-13.48	-13.49	0.05	416.38	-	-	-
128	VAE	-13.37	-13.39	-13.42	0.02	604.04s	0.37	0.056	0.25
	LMF	-13.39	-13.42	-13.45	0.03	241.14s	0.28	0.006	0.0065
	NN	-13.47	-13.50	-13.45	0.02	490.40s	-	-	-
	NN Full	-13.47	-13.47	-13.47	0.00	1431.30s	-	-	-

Table 11.1: VSD: the best is the closest to 0, MPR: the best is the lowest, NDCG: the greatest is the best, Recall: the greatest is the best. The training time includes the time required to continuously calculate the different metrics. The training process was achieved on a MacBook Pro 2021 - Apple M1 Pro - 16GB.

11.2 Model selection

We selected the best model by picking the one which gives the best VSD validation score, we then compute all the other metrics based on this model. To retrieve an accurate list of recommended items to calculate the VSD we perform the following operations:

1. Retrieve the user's tracks sorted by their affinity
2. Perform recommendation with the model (NN, LMF or VAE), retrieving the items list with a predicted affinity (which is the distance for the NN)
3. We sort recommendations list by the score we got

4. We remove the first $N = 25$ items of ground truth from the recommendations
5. We calculate the similarity score of the first $N = 25$ items of the sorted ground truth and the top $N = 25$ items of the recommendation

11.3 NN vs NN Full

As part of the data ablation analysis we separated our Nearest Model fitted only on the train set and the same model fitted with the full dataset including the train set, the validation and the test set. This allows us to determine if we get better results by adding more data in the Nearest Neighbors space or not.

11.4 Best Nearest Neighbor hyper-parameters

As expected and discussed in Section 6.1 the Grid Search (training) of Nearest Neighbor model always conclude with conservative hyper-parameters. With a low `n_neighbors` and a low `radius`. It is very natural since our VSD metric is based on the feature space.

11.5 Discussion

We can see that for every dataset size, except the eight users one, the collaborative systems perform better. We also noticed that the time required to provide inference by the Nearest Neighbors model increases significantly when increasing the number of users (because the number of tracks also increases) and after a certain amount of users it is not even possible to perform a Grid Search so we picked the best optimal hyper-parameters we got for the hyper-parameters with a smaller user dataset, which are: `n_neighbors=3`, `radius=0`, `metric=cosine`.

8 users

For the smallest dataset size, as we expected during our discussion in Section 6.1, the Nearest Neighbors model yields the best training results. However, it performs a bit worse than the LMF in generalization capacity when performing recommendation for users it doesn't know. For the MPR metric we refer to the MPR given by the model which gave the best VSD metric and they provide both really close results, but the VAE yields the best MPR score.

16 users

For 16 users, the LMF provides better results on the train set but perform worse in generalization capabilities than the Nearest Neighbors. For the MPR the VAE has still a better score than the LMF.

32, 64, 128 users

For higher number of users we noticed that the VAE takes the lead for both training results and generalization capabilities. Focusing on the MPR, we can see that it doesn't necessarily provide the best ordered recommendation, the MPR value gets better for the LMF. The Nearest Neighbors starts to have an enormous amount of items in it's space which leads to significant inference time that isn't compatible to with online recommender system that should be scalable.

We can see that the "test VSD score" for the VAE varies less for different dataset sizes than the LMF or the Nearest Neighbors, leading to more result stability, which is a great advantage to have more confidence when training the model with different amount of users. Although the LMF is the model which has the best training time and that increases the least, this is probably due to the fact that the gradient update is clearly defined and does not rely on algorithmic calculations. In conclusion we can say that adopting a collaborative recommender systems for a small user base isn't necessarily a bad idea and can lead to high quality recommendations.

12. Future perspectives and improvements

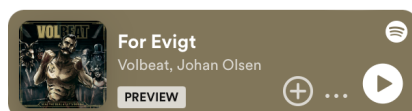
12.1 Sparse matrix support for large batches

As mentioned in 7.2.4 our matrix are highly sparse, meaning that our entries are mostly zeros. Multiple Python libraries implement tools to work with sparse matrix and allow performing operations with them. To keep our implementations simple we wanted to use PyTorch only, which also support sparse matrices but their implementation is currently on beta and cannot be run on the devices we used (CPU/MPS). Since our scientific question involves small datasets we didn't spend more time implementing a sparse matrix support. For a production purpose, it would be a very important aspect to implement, for instance in the current state we cannot load and use the datasets Liang et al. (2018) used, it simply crashed due to insufficient memory when loading the matrix in a dense format.

12.2 Human based metrics

In the case where we were allowed by Spotify to use their data to train a recommender in an educational purpose, we could evaluate our model with an app our users can access and rate the recommendation made by our systems. We have made a prototype with dummy data without connecting that to any model since we aren't allowed to train them with Spotify data.

You are logged as: owengombas



How do you feel about this song?

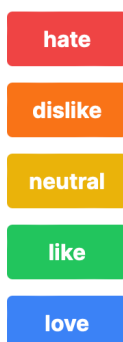


Figure 12.1: human-based-metric.png - Our users are recommended a song from Spotify and can rate them with ordered qualitative feedback

12.3 Individual improvements of the models

To be able to compare our models with more correctness we enforced a common baseline on them to see how well they perform in **the same conditions**. But this context might not yield the optimal result for every model. For instance we could find the optimal number of latent features for the collaborative models.

12.4 Metric compatibility

As discussed in the Sections [8.5](#) and [9.5](#) we may not succeed porting the LMF metrics to the VAE and the VAE ones to the LMF. Further investigations should be done to be able to compare them successfully. However we did succeed implementing the evolution metrics of the LMF for the LMF and the VAE ones for the VAE, allowing us to evaluate them individually.

13. Developer documentation

We generated a documentation from the doc-strings of the functions and classes we implemented. This documentation is in HTML format and can be viewed with any web browser by opening the `index.html` file. It will be submitted along with this report.

recommende r-systems

Navigation

Contents:

[lib package](#)
[recommenders package](#)

Quick search

Welcome to recommender- systems's documentation!

Contents:

- [lib package](#)
 - [Submodules](#)
 - [lib.artificial_data module](#)
 - [lib.dimensionality_reduction module](#)
 - [lib.genre_normalizer module](#)
 - [lib.matrix_dataset module](#)
 - [lib.plotting module](#)
 - [lib.preprocessing module](#)
 - [lib.spoti module](#)
 - [lib.user_similarities module](#)
 - [Module contents](#)
- [recommenders package](#)
 - [Submodules](#)
 - [recommenders.lmf module](#)
 - [recommenders.metrics module](#)
 - [recommenders.nearest_neighbours module](#)
 - [recommenders.vae module](#)
 - [Module contents](#)

Indices and tables

- [Index](#)
- [Module Index](#)
- [Search Page](#)

©2024, Owen Gombas, Arthur Gyga. | Powered by [Sphinx 5.0.2](#) & [Alabaster 0.7.12](#) | [Page source](#)

Figure 13.1: doc.png - How the documentation should look like in your web browser

14. Conclusion

During this project, we implemented and went into the details of three different recommender systems for Spotify songs that are based on two different paradigms: We started with content-based filtering using a nearest neighbors model (cf. Chapter 6) and followed with collaborative filtering using on the one hand a Logistic Matrix Factorization (cf. Chapter 8) and on the other hand Variational Autoencoders (cf. Chapter 9).

With our objective being to be able to measure and compare the effectiveness of content-based filtering and collaborative filtering in recommender systems while dealing with a small user base. We implemented a metric: the Vector Set Divergence outlined in Section 5.1 to be able to quantitatively compare the different outputs of all our models. Additionally, we also implemented a data generator as discussed in Section 2.2, to address the size limitation of the dataset we retrieved from Spotify, and have the flexibility to control the quantity of data and improve our experiment.

Our results as detailed in Chapter 11 show that, based on the Vector Set Divergence metric, the collaborative-filtering paradigm despite the constraint of a small user base achieves in general better scores, with the VAE and LMF models scoring very similar scores, than the content-based filtering. It is also worth noting that the Nearest Neighbors model tend to have very slow training times compared to both other models.

In future work, investigating hybrid approaches that combine both content-based filtering and collaborative-filtering may increase the accuracy and inference time. Adding a human-based metric could also enhance the overall accuracy of the models and ensure the models serve their purpose.

15. Bibliography

- Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE International Conference on Data Mining*, pages 263–272, Dec 2008. doi: 10.1109/ICDM.2008.22.
- Christopher C. Johnson. Logistic matrix factorization for implicit feedback data. In *NIPS 2014 Workshop on Music, Machine Learning, and Metadata*, 2014. URL <https://web.stanford.edu/~rezab/nips2014workshop/submits/logmat.pdf>.
- Christopher C. Johnson. implicit-mf. <https://github.com/MrChrisJohnson/implicit-mf>, 2016.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022.
- Dawen Liang. vae_cf. https://github.com/dawenl/vae_cf, 2018.
- Dawen Liang, Rahul G. Krishnan, Matthew D. Hoffman, and Tony Jebara. Variational autoencoders for collaborative filtering, 2018.
- Andriy Mnih and Russ R Salakhutdinov. Probabilistic matrix factorization. In J. Platt, D. Koller, Y. Singer, and S. Roweis, editors, *Advances in Neural Information Processing Systems*, volume 20. Curran Associates, Inc., 2007. URL https://proceedings.neurips.cc/paper_files/paper/2007/file/d7322ed717dedf1eb4e6e52a37ea7bcd-Paper.pdf.
- Arvind Narayanan and Vitaly Shmatikov. Robust de-anonymization of large sparse datasets. In *2008 IEEE Symposium on Security and Privacy (sp 2008)*, pages 111–125, 2008. doi: 10.1109/SP.2008.33.
- Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In Eric P. Xing and Tony Jebara, editors, *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pages 1278–1286, Beijing, China, 22–24 Jun 2014. PMLR. URL <https://proceedings.mlr.press/v32/rezende14.html>.
- Younggyo Seo. vae-cf-pytorch. <https://github.com/younggyoseo/vae-cf-pytorch>, 2021.